

PAVELOR: A High-Security Hash Algorithm Optimized for Software Implementations

Ruichen Wu, Yufei Yuan, Danxun Zhang, Shanpeng Wei, Wenling Wu

Institute of Software, Chinese Academy of Sciences, Beijing 100190, China

University of Chinese Academy of Sciences, Beijing 100049, China

Zhongguancun Laboratory, Beijing, China

Contents

1	Specification of PAVELOR	4
1.1	Notation and Operational Conventions	4
1.2	Algorithm Specification	5
1.2.1	Message Padding	6
1.2.2	State Initialization	6
1.2.3	Message Absorption	7
1.2.4	Digest Generation	8
1.3	The Permutation PAFF	9
2	Design Rationale of PAVELOR	11
2.1	The Sponge Construction	11
2.2	AES Instructions	11
2.3	State Size and Rate/Capacity Parameter Selection	12
2.4	Underlying Permutation Design	13
2.4.1	Permutation Designs Based on AES Instructions	13
2.4.2	Design of the Permutation PAFF	15
2.5	Parameter Selection	19
2.5.1	Selection of wt(GammaR)	19
2.5.2	Selection of p1, p2, and GammaR	21
3	Security of PAVELOR	23
3.1	Security Claims	23
3.1.1	Classical Security Claims	23
3.1.2	Quantum Security Claims	23
3.2	Security of the Underlying Permutation	24
3.2.1	Differential and Linear Attacks	24
3.2.2	Impossible Differential Attacks	24
3.2.3	Integral Attacks	25
3.2.4	Exchange-Equivalence Attacks	25
3.3	Security of the Hash Algorithm	28
3.3.1	Resistance to Length-Extension Attacks	28
3.3.2	Meet-in-the-Middle Attacks	28
3.3.3	Rebound Attacks	32
4	Performance of PAVELOR	34
4.1	x86 Platform Performance Tests	34
4.2	Hygon Platform Performance Test Results	37

5	Features of PAVELOR	38
6	Statistical Tests of PAVELOR	39
6.1	Statistical Test Results for PAVELOR-512	39
6.2	Statistical Test Results for PAVELOR-768	39
6.3	Statistical Test Results for PAVELOR-1024	40
7	Test Vectors of PAVELOR	41
7.1	PAVELOR-512	41
7.2	PAVELOR-768	43
7.3	PAVELOR-1024	44

1 Specification of PAVELOR

1.1 Notation and Operational Conventions

The following notation and operational conventions are used throughout the algorithm specification:

- b : the state width, with $b = 2560$ bits.
- r : the rate of the sponge construction.
- c : the capacity of the sponge construction.
- M : a message of arbitrary length.
- h : the digest length, where $h = 512/768/1024$.
- S_i : the state $S_i = (S_{i,0}, \dots, S_{i,19})$. It has length $b = 2560$ bits and is divided into twenty 128-bit subblocks. The symbol $S_{i,j}$ denotes the j -th subblock in round i .
- RC : the round constant; RC_i denotes the round constant used in round i .
- $|X|$: the bit length of X .
- $X||Y$: the concatenation of X and Y .
- $X \oplus Y$: bitwise addition modulo 2, equivalently the bitwise XOR of X and Y .
- $\langle X \rangle_n$: the n -bit big-endian binary representation of the integer X , where $0 \leq X < 2^n$.
- $\lfloor X \rfloor_n$: for an arbitrary bit string $X = x_0x_1 \dots x_{L-1}$, define

$$\lfloor X \rfloor_n = x_0x_1 \dots x_{n-1},$$

i.e., the first n bits of X .

- $\text{AESENC}(X, Y)$: the AES encryption-round transformation with data input X and XOR input Y . It is defined as

$$\text{AESENC}(X, Y) = \text{MixColumns}(\text{ShiftRows}(\text{SubBytes}(X))) \oplus Y.$$

The operations MixColumns , ShiftRows , and SubBytes are those specified for AES.

- $A(X)$: the AES round transformation without the final round-key XOR, defined as

$$A(X) = \text{MixColumns}(\text{ShiftRows}(\text{SubBytes}(X))).$$

1.2 Algorithm Specification

The hash algorithm PAVELOR supports digest lengths of 512, 768, and 1024 bits. The corresponding instances are denoted by PAVELOR-512, PAVELOR-768, and PAVELOR-1024, respectively. PAVELOR adopts the sponge construction, as illustrated in Fig. 1, where r is the rate, c is the capacity, and h is the digest length. During the absorbing phase, the padded message is divided into blocks of length r , each of which is XORed into the first r bits of the state and followed by an invocation of the permutation PAFF. During the squeezing phase, the first r bits of the state are extracted and concatenated until the required digest length is reached.

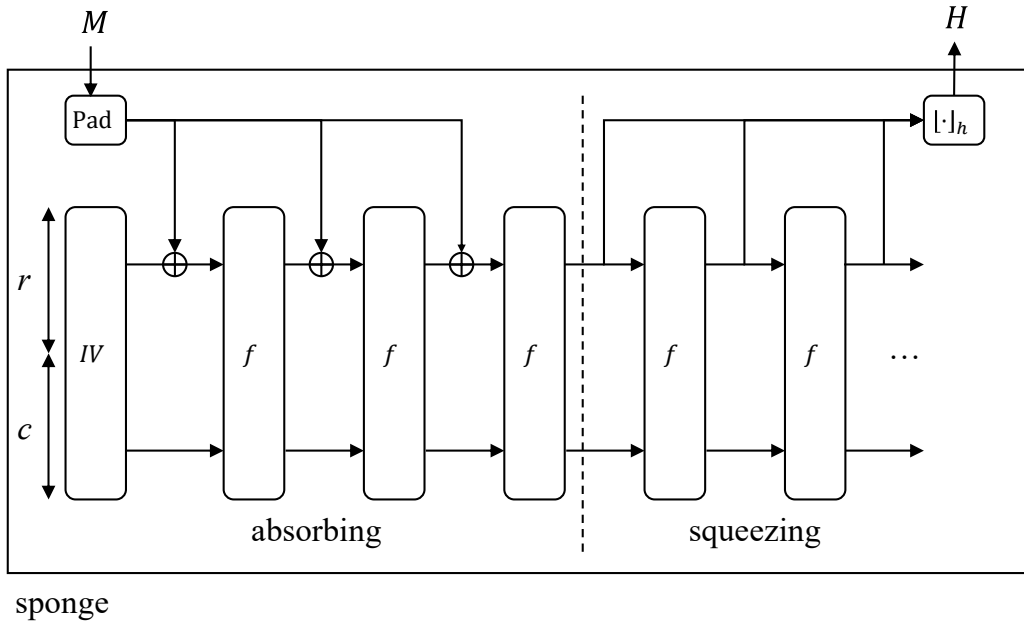


Figure 1: Sponge construction

Table 1 gives the choices of r and c for the different digest lengths of PAVELOR.

Table 1: Parameters of the sponge construction

Algorithm	State width b	Digest length h	Rate r	Capacity c
PAVELOR-512	2560	512	1536	1024
PAVELOR-768	2560	768	1024	1536
PAVELOR-1024	2560	1024	512	2048

The procedure for PAVELOR- h is as follows:

Algorithm 1 PAVELOR- $h(M)$ **Require:** Message bit string M **Ensure:** Hash output bit string H of length h

- 1: $M^* \leftarrow \text{Pad}(M, r)$ ▷ Message padding
- 2: $S \leftarrow \text{InitState}(h)$ ▷ State initialization
- 3: $S \leftarrow \text{Absorb}(S, M^*, r)$ ▷ Absorbing phase
- 4: $H \leftarrow \text{Squeeze}(S, h, r)$ ▷ Squeezing phase
- 5: **return** H

1.2.1 Message Padding

To align the message length to an integral number of r -bit blocks for absorption, PAVELOR uses the following deterministic and prefix-unambiguous padding rule. Given an arbitrary message $M \in \{0, 1\}^*$, the padded message is

$$\text{Pad}(M, r) = M \parallel 1 \parallel 0^k \parallel 1,$$

where k is the smallest nonnegative integer satisfying $|M| + 1 + k \equiv r - 1 \pmod{r}$. The resulting padded bit string has length divisible by r .

Algorithm 2 Pad(M, r)**Require:** Message M , rate r **Ensure:** Padded message M^*

- 1: $M_1 \leftarrow M \parallel 1$
- 2: $k \leftarrow (r - 1 - (|M_1| \bmod r)) \bmod r$
- 3: $M^* \leftarrow M_1 \parallel 0^k \parallel 1$
- 4: **return** M^*

1.2.2 State Initialization

For the instance with digest length h , the corresponding rate r , capacity c , and initialization vector are specified as

$$IV_h = (IV_{h,0}, IV_{h,1}, \dots, IV_{h,19}) \in (\{0, 1\}^{128})^{20}.$$

Let $\langle x \rangle_{128}$ denote the 128-bit big-endian encoding of the nonnegative integer x . The initialization vector is defined as

$$IV_h = \langle r \rangle_{128} \parallel \langle c \rangle_{128} \parallel \langle b \rangle_{128} \parallel \langle h \rangle_{128} \parallel 0^{128} \parallel \dots \parallel 0^{128}.$$

The initialization vectors for the different output lengths are shown in Table 2.

Table 2: Initialization vectors for the algorithm instances

Algorithm instance	Initialization vector IV_h
PAVELOR-512	$IV_{512,0} = \text{0x00000000000000000000000000600},$ $IV_{512,1} = \text{0x00000000000000000000000000400},$ $IV_{512,2} = \text{0x00000000000000000000000000A00},$ $IV_{512,3} = \text{0x00000000000000000000000000200},$ $IV_{512,i} = 0^{128}, \quad 4 \leq i \leq 19.$
PAVELOR-768	$IV_{768,0} = \text{0x00000000000000000000000000400},$ $IV_{768,1} = \text{0x00000000000000000000000000600},$ $IV_{768,2} = \text{0x00000000000000000000000000A00},$ $IV_{768,3} = \text{0x00000000000000000000000000300},$ $IV_{768,i} = 0^{128}, \quad 4 \leq i \leq 19.$
PAVELOR-1024	$IV_{1024,0} = \text{0x00000000000000000000000000200},$ $IV_{1024,1} = \text{0x00000000000000000000000000800},$ $IV_{1024,2} = \text{0x00000000000000000000000000A00},$ $IV_{1024,3} = \text{0x00000000000000000000000000400},$ $IV_{1024,i} = 0^{128}, \quad 4 \leq i \leq 19.$

During initialization, the initialization vector IV_h associated with the digest length h is selected, and the state subblocks are set by

$$S_j \leftarrow IV_{h,j}, \quad 0 \leq j < 20.$$

The resulting state S is used as the input state to the absorbing phase.

Algorithm 3 InitState(h)

Require: Digest length $h \in \{512, 768, 1024\}$

- ```

1: for $j \leftarrow 0$ to 19 do
2: $S_j \leftarrow IV_{h,j}$
3: end for
4: return S

```

### 1.2.3 Message Absorption

Let the padded message be  $M^* = \text{Pad}(M, r)$ , with length  $|M^*| = \ell \cdot r$ . It is divided into  $\ell$  blocks of  $r$  bits:

$$M^* = M^{(0)} \parallel M^{(1)} \parallel \dots \parallel M^{(\ell-1)}, \quad M^{(i)} \in \{0, 1\}^r.$$

The first  $r$  bits of the state  $S$  are denoted by the rate part  $S_{\text{rate}}$ , and the remaining  $c$  bits by the capacity part  $S_{\text{cap}}$ :

$$S = (S_{\text{rate}}, S_{\text{cap}}), \quad |S_{\text{rate}}| = r, \quad |S_{\text{cap}}| = c.$$

Starting from the initial state  $S$ , the absorbing phase processes each message block  $M^{(i)}$ ,  $i = 0, 1, \dots, \ell - 1$ , as follows:

1.  $S_{\text{rate}} \leftarrow S_{\text{rate}} \oplus M^{(i)}$ ;
2. set  $S \leftarrow \text{PAFF}(S)$ ;

After all message blocks have been processed, the resulting state  $S$  is used as the input to the squeezing phase.

---

**Algorithm 4** Absorb( $S, M^*, r$ )

---

**Require:** Initial state  $S$ , padded bit string  $M^*$ , rate  $r$

**Ensure:** State  $S$  after all message blocks have been absorbed

- 1: Partition  $M^*$  into  $\ell$  blocks  $M^{(0)}, \dots, M^{(\ell-1)}$  of length  $r$  ▷ Partition the message according to rate  $r$
  - 2: **for**  $i \leftarrow 0$  **to**  $\ell - 1$  **do**
  - 3:      $S_{\text{rate}} \leftarrow S_{\text{rate}} \oplus M^{(i)}$  ▷ XOR the  $i$ -th message block into the rate part
  - 4:      $S \leftarrow \text{PAFF}(S)$  ▷ Update state  $S$
  - 5: **end for**
  - 6: **return**  $S$  ▷ Return the state after absorption
- 

### 1.2.4 Digest Generation

Let the hash output length be  $h$  bits. In the digest generation phase, the permutation is iteratively applied to the state obtained after message absorption, and output bits are read from the rate part.

Given the post-absorption state  $S$  and the target output length  $h$ , initialize the output bit string as  $H \leftarrow \varepsilon$ . Repeat the following steps:

1. let  $T$  be the rate part  $S_{\text{rate}}$  of the current state;
2. append the first  $\min(r, h - |H|)$  bits of  $T$  to  $H$ ;
3. if  $|H| \geq h$ , terminate; otherwise, set  $S \leftarrow \text{PAFF}(S)$  and repeat the procedure

Finally, the hash value is defined as

$$\text{PAVELOR-}h(M) = \lfloor H \rfloor_h.$$

---

**Algorithm 5** Squeeze( $S, h, r$ )

---

**Require:** State  $S$ , digest length  $h$ , rate  $r$

**Ensure:** Digest  $H$  of length  $h$

- 1:  $H \leftarrow \varepsilon$  ▷ Initialize the digest as the empty string
  - 2: **while**  $|H| < h$  **do**
  - 3:      $T \leftarrow S_{\text{rate}}$  ▷ Read the rate part of the current state
  - 4:      $H \leftarrow H || \lfloor T \rfloor_{\min(r, h - |H|)}$  ▷ Append bits according to the remaining digest length
  - 5:     **if**  $|H| \geq h$  **then**
  - 6:         **break** ▷ Terminate once the required digest length has been reached
  - 7:     **else**
  - 8:          $S \leftarrow \text{PAFF}(S)$  ▷ Update state  $S$
  - 9:     **end if**
  - 10: **end while**
  - 11: **return**  $\lfloor H \rfloor_h$  ▷ Return the first  $h$  bits as the final digest
-

### 1.3 The Permutation PAFF

The permutation PAFF is defined as

$$\text{PAFF} : (\{0, 1\}^{128})^{20} \rightarrow (\{0, 1\}^{128})^{20}.$$

Both its input and output are states consisting of twenty 128-bit subblocks. Let  $\mathbb{V} = (\{0, 1\}^{128})^{20}$ . Then PAFF is a 24-round iterative permutation over the state space  $\mathbb{V}$  and can be written as

$$\text{PAFF}(\cdot) = \rho_{23} \circ \rho_{22} \circ \cdots \circ \rho_0(\cdot),$$

where  $\rho_i$  denotes the round mapping in round  $i$ .

To describe the round mapping  $\rho_i$ , define the index mappings  $p_1$  and  $p_2$  as follows:

$$\begin{aligned} p_1 &= (19, 4, 6, 17, 9, 0, 13, 11, 8, 5, 15, 10, 16, 3, 12, 14, 18, 7, 2, 1), \\ p_2 &= (1, 0, 3, 4, 5, 2, 7, 8, 9, 10, 6, 12, 13, 14, 15, 16, 17, 18, 19, 11). \end{aligned}$$

Let the input and output states of round  $i$  be

$$S_i = (S_{i,0}, S_{i,1}, \dots, S_{i,19}) \in \mathbb{V}, \quad S_{i+1} = (S_{i+1,0}, S_{i+1,1}, \dots, S_{i+1,19}) \in \mathbb{V}.$$

Thus  $S_{i+1} = \rho_i(S_i)$ . The  $j$ -th subblock of the round mapping in round  $i$  is given by

$$S_{i+1,j} = \begin{cases} A(S_{i,p_1(j)}) \oplus S_{i,p_2(j)}, & j \neq 7, \\ RC_i \oplus S_{i,8}, & j = 7. \end{cases}$$

Each round of PAFF can be interpreted as follows. First, all subblocks of the input state  $S_i$  except the round-constant injection position are processed in parallel by one AES round transformation without round-key XOR; at the round-constant injection position, this local transformation is replaced by the round constant, yielding an intermediate state. Then, according to the mappings  $p_1$  and  $p_2$ , the corresponding subblocks are selected from this intermediate state and from the input state  $S_i$ , respectively, and are XORed subblock by subblock to obtain the next state  $S_{i+1}$ . The resulting round implementation is specified below.

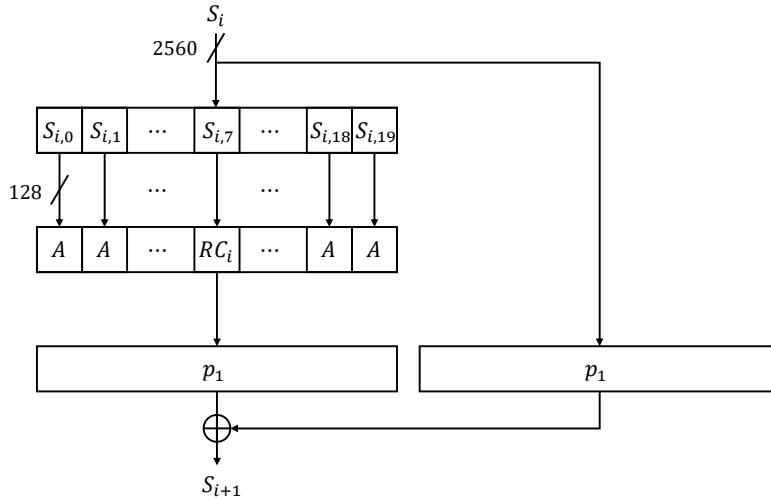


Figure 2: Round function of PAFF. The symbol  $A$  denotes the AES round transformation without the round-key XOR.

**Algorithm 6** PAFF ( $S$ )**Require:** Input state  $S_0 = (S_{0,0}, S_{0,1}, \dots, S_{0,19})$ **Ensure:** Output state  $S_{24}$ 


---

```

1: for $i \leftarrow 0$ to 23 do
2: $S_{i+1,0} \leftarrow \text{AESENC}(S_{i,19}, S_{i,1})$
3: $S_{i+1,1} \leftarrow \text{AESENC}(S_{i,4}, S_{i,0})$
4: $S_{i+1,2} \leftarrow \text{AESENC}(S_{i,6}, S_{i,3})$
5: $S_{i+1,3} \leftarrow \text{AESENC}(S_{i,17}, S_{i,4})$
6: $S_{i+1,4} \leftarrow \text{AESENC}(S_{i,9}, S_{i,5})$
7: $S_{i+1,5} \leftarrow \text{AESENC}(S_{i,0}, S_{i,2})$
8: $S_{i+1,6} \leftarrow \text{AESENC}(S_{i,13}, S_{i,7})$
9: $S_{i+1,7} \leftarrow RC_i \oplus S_{i,8}$
10: $S_{i+1,8} \leftarrow \text{AESENC}(S_{i,8}, S_{i,9})$
11: $S_{i+1,9} \leftarrow \text{AESENC}(S_{i,5}, S_{i,10})$
12: $S_{i+1,10} \leftarrow \text{AESENC}(S_{i,15}, S_{i,6})$
13: $S_{i+1,11} \leftarrow \text{AESENC}(S_{i,10}, S_{i,12})$
14: $S_{i+1,12} \leftarrow \text{AESENC}(S_{i,16}, S_{i,13})$
15: $S_{i+1,13} \leftarrow \text{AESENC}(S_{i,3}, S_{i,14})$
16: $S_{i+1,14} \leftarrow \text{AESENC}(S_{i,12}, S_{i,15})$
17: $S_{i+1,15} \leftarrow \text{AESENC}(S_{i,14}, S_{i,16})$
18: $S_{i+1,16} \leftarrow \text{AESENC}(S_{i,18}, S_{i,17})$
19: $S_{i+1,17} \leftarrow \text{AESENC}(S_{i,7}, S_{i,18})$
20: $S_{i+1,18} \leftarrow \text{AESENC}(S_{i,2}, S_{i,19})$
21: $S_{i+1,19} \leftarrow \text{AESENC}(S_{i,1}, S_{i,11})$
22: end for
23: return S_{24}

```

---

The round constants  $RC_i$  are derived from the binary expansion of the fractional part of  $\pi = 3.1415926\dots$ . For each round index  $i \in \{0, 1, \dots, 23\}$ , the corresponding 128-bit round constant  $RC_i$  is given in Table 3.

Table 3: Round constants

| $i$ | $RC_i$                              | $i$ | $RC_i$                             |
|-----|-------------------------------------|-----|------------------------------------|
| 0   | 0x243f6a8885a308d313198a2e03707344  | 12  | 0x6c9e0e8bb01e8a3ed71577c1bd314b27 |
| 1   | 0xa4093822299f31d0082efa98ec4e6c89  | 13  | 0x78af2fda55605c60e65525f3aa55ab94 |
| 2   | 0x452821e638d01377be5466cf34e90c6c  | 14  | 0x5748986263e8144055ca396a2aab10b6 |
| 3   | 0xc0ac29b7c97c50dd3f84d5b5b5470917  | 15  | 0xb4cc5c341141e8cea15486af7c72e993 |
| 4   | 0x9216d5d98979fb1bd1310ba698dfb5ac  | 16  | 0xb3ee1411636fbc2a2ba9c55d741831f6 |
| 5   | 0x2ffd72dbd01adfb7b8e1afed6a267e96  | 17  | 0xce5c3e169b87931eafd6ba336c24cf5c |
| 6   | 0xba7c9045f12c7f9924a19947b3916cf7  | 18  | 0x7a325381289586773b8f48986b4bb9af |
| 7   | 0x801f2e2858efc16636920d871574e690  | 19  | 0xc4bfe81b6628219361d809ccfb21a991 |
| 8   | 0xa458fea3f4933d7e0d95748f728eb658  | 20  | 0x487cac605dec8032ef845d5de98575b1 |
| 9   | 0x718bcd5882154aee7b54a41dc25a59b5  | 21  | 0xdc262302eb651b8823893e81d396acc5 |
| 10  | 0x9c30d5392af26013c5d1b023286085f0  | 22  | 0xf6d6ff383f442392e0b4482a48420040 |
| 11  | 0xca417918b8db38ef8e79dcdb0603a180e | 23  | 0x69c8f04a9e1f9b5e21c66842f6e96c9a |

## 2 Design Rationale of PAVELOR

### 2.1 The Sponge Construction

PAVELOR adopts the sponge construction as its overall framework, primarily because of its simplicity, extensibility, and mature generic-security theory. As a permutation-based hash construction, the sponge framework concentrates the generic security margin in the capacity parameter and makes the structural security of the underlying permutation the main object of analysis. This layered viewpoint is consistent with the design philosophy of Keccak/SHA-3 and leads to a clear path for parameter selection and security justification.

In the classical security model, sponge constructions have been supported by systematic provable-security results. Bertoni et al. showed that, when the underlying function is modeled as a random transformation or a random permutation, the sponge construction can approach a random oracle in the indistinguishability sense, with the advantage mainly controlled by the capacity parameter [8]. The Keccak design document further formulates this idea as the hermetic sponge strategy: if the underlying permutation has no exploitable structural weakness, then an attack that significantly exceeds the capacity bound should be attributed to a structural distinguisher of the underlying permutation rather than to a generic flaw of the sponge framework itself [9]. Therefore, using the sponge framework allows the security argument for PAVELOR to be decomposed explicitly into a generic capacity bound and the structural security of the underlying permutation PAFF.

The sponge construction also provides strong functional extensibility and fits well with standardized interfaces. Fixed-length hash outputs, instances at different security levels, and extendable-output functions can all be derived naturally within the same framework; the SHA-3 standard, for example, defines both the fixed-output SHA3- $d$  family and the extendable-output SHAKE family from the Keccak sponge construction [27]. For PAVELOR, this means that PAVELOR-512, PAVELOR-768, and PAVELOR-1024 can be derived from the same underlying permutation PAFF through different parameter choices, thereby reducing design complexity and keeping the security analysis and software implementation consistent across instances.

In addition, sponge-like constructions have also been studied in the quantum-query model. Existing work analyzes the post-quantum security implications of sponge constructions from the viewpoints of collapsing properties, quantum collision resistance, and quantum indistinguishability [13, 2]. These results indicate that the sponge framework has a mature generic-security foundation not only in the classical model, but also as a reference point for parameter selection under quantum attack models. The security claims made below for PAVELOR are still based on the concrete output lengths, capacity parameters, and structural analysis of the underlying permutation PAFF.

### 2.2 AES Instructions

PAVELOR uses AES instructions as the basic building blocks of its underlying permutation. This choice is not intended as a platform-specific micro-optimization. Rather, it is motivated by the broad hardware support for AES instructions in mainstream general-purpose processors. On x86 platforms, Intel AES-NI provides dedicated instructions for AES operations and can efficiently execute AES round transformations on 128-bit data blocks. AMD64 processors implementing the AES instruction extensions likewise provide

hardware acceleration for AES operations. On Arm platforms, the Armv8/Armv9 Cryptographic Extension provides instructions such as AESE, AESD, AESMC, and AESIMC, and supports both AArch64 and AArch32 execution states. AES instructions have therefore become a common hardware capability across the two dominant instruction-set families, x86 and Arm. Building the permutation from AES instructions therefore supports efficient and portable software implementations [20, 1, 4].

In addition to their broad hardware availability, the structural properties of AES instructions make them suitable as local transformation units in a large-state permutation. First, their natural processing granularity is 128 bits. This allows the large state to be regularly divided into fixed-size subblocks, simplifying state organization, parallel scheduling, and implementation description. Second, an AES round transformation contains a well-studied nonlinear layer and an intra-block diffusion layer, and therefore provides efficient local mixing within a single 128-bit subblock. Accordingly, in a large-state permutation, the AES round transformation can be used as a standardized local operation, while the main design focus can be placed on the coupling relations and cross-subblock diffusion mechanisms among different subblocks. This layered design exploits the hardware acceleration already available for AES round operations and preserves regularity, analyzability, and implementation efficiency.

Overall, AES instructions provide a unified 128-bit local transformation granularity for large-state permutations, together with a mature implementation basis on mainstream x86 and Arm platforms. Using this hardware-friendly local primitive, a large-state permutation design can strike a favorable balance among implementation efficiency, structural regularity, cross-platform portability, and clarity of subsequent security analysis.

## 2.3 State Size and Rate/Capacity Parameter Selection

PAVELOR adopts the sponge construction, whose internal state width satisfies  $b = r + c$ , where the rate  $r$  determines the number of bits that can be absorbed or squeezed in each permutation call, and the capacity  $c$  determines the generic security bound of the sponge construction [8, 9].

The capacity parameter is first determined by the security target associated with the digest length. Let  $h$  denote the digest length. For an  $h$ -bit digest, the collision-security bound of an ideal hash function is  $2^{h/2}$ , while the preimage and second-preimage bounds are  $2^h$ . To prevent the capacity of the sponge construction from becoming the limiting factor for security, this design follows a conservative capacity-selection principle similar to SHA-3, namely  $c = 2h$  [27]. Hence, for the highest digest length  $h = 1024$ , the capacity is set to  $c = 2048$ .

Once the capacity is fixed, selecting the state width  $b$  is equivalent to selecting the rate  $r = b - c$ . A larger rate reduces the number of permutation calls for long messages, but it also increases the number of 128-bit subblocks maintained and updated by the underlying permutation. The state width is therefore chosen by balancing capacity margin, sponge-interface throughput, and permutation implementation cost.

The underlying permutation uses the AES round function on 128-bit subblocks as its basic operation. On AES-NI-capable platforms, high-throughput AES implementations typically schedule multiple 128-bit data blocks in order to exploit the pipelined execution of AES round instructions. Intel’s AES-NI implementation guidance indicates that four-block and eight-block parallelism are typical optimization granularities, whereas excessive

pipelining may fail to improve performance and can increase register pressure [20]. It should be emphasized that the sponge rate  $r$  is not the same as the AES instruction parallelism; in the absorbing phase, the message block is only XORed into the rate part. Nevertheless, the rate determines how much message data can be amortized over one complete permutation call. This design therefore takes four 128-bit subblocks as the minimum effective rate granularity for the highest-security instance, namely  $r_{\min} = 4 \cdot 128 = 512$ .

Combining the capacity requirement  $c = 2048$  of the highest-security instance with the minimum rate requirement  $r_{\min} = 512$  gives the smallest state width satisfying both constraints, namely  $b = 2048 + 512 = 2560$ . This choice enables the 1024-bit digest instance to satisfy the  $c = 2h$  capacity principle while still processing 512 bits of message data per permutation call, thereby preventing an excessively small rate from becoming the dominant performance bottleneck at the sponge interface.

On the other hand, the state width should not be increased without limit. If  $b$  were further increased, the rate of the highest-security instance could also be increased; however, the number of 128-bit subblocks, subblock-coupling relations, and temporary variables inside the permutation would increase accordingly, bringing higher register pressure, data movement, and scheduling complexity. Considering the AES-NI implementation guidance on parallel granularity and register resources, this design selects  $b = 2560$  as a compromise among capacity margin, minimum effective rate, and implementation complexity [20, 23].

Under a unified internal permutation and a unified state width  $b = 2560$ , the different digest-length instances are obtained by adjusting the rate/capacity partition:

$$\begin{aligned} h = 512 : \quad & (b, c, r) = (2560, 1024, 1536), \\ h = 768 : \quad & (b, c, r) = (2560, 1536, 1024), \\ h = 1024 : \quad & (b, c, r) = (2560, 2048, 512). \end{aligned}$$

All three instances satisfy  $c = 2h$ . The 1024-bit instance uses the largest capacity while retaining a 512-bit rate; the shorter-digest instances have larger rates under the same state width and therefore achieve higher absorbing and squeezing throughput.

## 2.4 Underlying Permutation Design

### 2.4.1 Permutation Designs Based on AES Instructions

Existing public permutation designs based on AES instructions or AES round functions can be broadly classified into two categories. The first category consists of SPN-based designs. Their basic idea is to apply AES round transformations in parallel to several 128-bit substates and to use an additional linear mixing layer to achieve diffusion across substates. Haraka [25] and related AES-like designs are representative of this direction. The second category consists of Feistel-based or generalized-Feistel-based designs. Their basic idea is to use the AES round function as a local update unit and to build a full-state permutation through branch updates, XOR feedback, and branch exchanges. Simpira v2 [21] and Areion [24] are representative examples of this direction.

**SPN-based designs and their limitations** SPN-based permutation designs typically provide fast diffusion [14, 15]. In such structures, provided that the cross-substate linear mixing layer has sufficiently strong diffusion, active bytes can usually propagate to the

entire state within a small number of rounds [14, 19, 12]. Consequently, the SPN approach is well suited to achieving strong diffusion properties.

However, in large-state permutation designs, this approach can also introduce a significant implementation burden. The AES round function itself mainly provides nonlinear confusion and local diffusion within a 128-bit subblock [28, 20]. To further achieve rapid coupling among different subblocks, an additional full-state linear mixing layer is typically required [25, 7, 12]. As the state width increases, this mixing layer often incurs high data-rearrangement cost, register-scheduling pressure, and implementation complexity, and may become the dominant cost in the entire permutation [7, 23]. Therefore, the main issue for the SPN approach in a large-state setting is not the construction of the local nonlinear component, but how to realize efficient full-state diffusion at low cost.

**Feistel-based designs and their limitations** Feistel-based designs take a different approach [35, 21]. Such structures usually update only part of the branches in each round and propagate state information gradually through XOR feedback and branch permutations, thereby avoiding an explicit heavy full-state linear layer in every round [35, 21]. The Feistel approach therefore provides favorable scalability in large-state designs and makes it easier to control the implementation cost of a single round.

This lightweight branch-update mechanism, however, also limits the diffusion speed. Since information propagation between subblocks relies on multiple rounds of gradual updates and cross-branch feedback, when the state consists of many 128-bit subblocks, a difference or activity in a single subblock often requires more rounds to propagate to the entire state [35, 11, 16, 34, 33]. In other words, the main advantage of the Feistel approach is its low per-round cost, while its main limitation is relatively slow cross-subblock diffusion, which may increase the total number of rounds required to reach the target security margin.

**Design approach** The two paradigms above illustrate a fundamental tradeoff in AES-based large-state permutation design. On the one hand, the design should exploit the efficient local nonlinearity and 128-bit internal diffusion provided by AES instructions. On the other hand, it must establish sufficiently fast coupling among multiple 128-bit subblocks, while avoiding an overly costly full-state linear mixing layer. Hence, the key design problem for the underlying permutation is not to reconstruct the local nonlinear transformation, but to achieve fast, regular, and analyzable cross-subblock diffusion at low implementation cost.

Recent public schemes aimed at high-performance software implementations show that AES-instruction-based state updates need not be restricted to traditional SPN or Feistel paradigms. In authenticated encryption, schemes such as AEGIS [39], Tiaoxin [29], and Rocca [32] demonstrate that multiple 128-bit state words can be organized as an internally parallel state, with a single AES round used as the basic local update component, and lightweight XOR coupling, message injection, and state rearrangement used to achieve effective cross-subblock propagation. The proposed permutation is inspired by this line of designs: it preserves the efficient local-update advantage of AES instructions, while accelerating cross-subblock diffusion through a lightweight state-level coupling mechanism, thereby balancing diffusion speed, implementation cost, and structural regularity.

### 2.4.2 Design of the Permutation PAFF

Motivated by the discussion above, an AES-instruction-based large-state permutation should satisfy two goals simultaneously. On the one hand, it should fully exploit the efficient local nonlinear transformation and intra-block diffusion provided by AES instructions at the 128-bit granularity. On the other hand, it should establish a fast and low-cost coupling mechanism among multiple 128-bit subblocks. Therefore, the design focus here is not to reconstruct a fine-grained nonlinear layer, but to improve the efficiency of information propagation among subblocks while avoiding a heavy full-state linear layer.

In existing large-state structures, Feistel-type schemes often avoid an explicit heavy full-state linear layer, but their cross-subblock diffusion typically relies on multi-round branch updates and gradual propagation. In particular, for Type-II generalized Feistel structures, when the state consists of  $k$  subblocks, propagating a difference from a single subblock to all  $k$  subblocks usually requires a number of rounds that grows linearly with  $k$  [35, 36]. When the state size is large, this gradual diffusion increases the number of rounds needed to achieve sufficient mixing, potentially leaving more room for impossible differential, saturation, and truncated differential analyses [36]. Reducing the number of rounds required for sufficient diffusion is therefore one of the important goals of the permutation design.

To quantitatively evaluate diffusion efficiency under different structural parameters, the analysis follows the characterization of diffusion rounds for generalized Feistel structures by Suzuki and Minematsu and introduces two metrics: full-subblock diffusion rounds and full-byte diffusion rounds [35]. The former measures the speed of information propagation among different 128-bit subblocks, while the latter further characterizes the number of rounds needed for byte-level information to diffuse to the entire state. Since the symbol  $r$  is already used for the sponge rate, the number of permutation rounds is denoted below by  $\ell$ .

Let  $\pi$  be a one-round permutation over a  $b$ -bit state, and let its  $\ell$ -round iteration be denoted by  $\pi^\ell$ . The diffusion rounds considered here are defined in terms of dependency: if the value of an output position depends on an input position, then the input position is said to influence that output position.

**Definition 2.1** (Full-subblock diffusion rounds). Let the state  $S_i = (S_{i,0}, S_{i,1}, \dots, S_{i,m-1})$  consist of  $m$  equal-length subblocks. If there exists a smallest positive integer  $\ell$  such that every input subblock  $S_{0,j}$  can influence all output subblocks  $S_{\ell,0}, S_{\ell,1}, \dots, S_{\ell,m-1}$  after  $\ell$  iterations  $\pi^\ell$ , then the permutation  $\pi$  is said to achieve full-subblock diffusion in  $\ell$  rounds. This minimum number of rounds is denoted by

$$D_{\text{sub}}(\pi) \triangleq \min \{ \ell \mid \forall j, k \in \{0, 1, \dots, m-1\}, S_{\ell,k} \text{ depends on } S_{0,j} \}.$$

**Definition 2.2** (Full-byte diffusion rounds). If  $b$  is a multiple of 8, the state  $S_i$  can be viewed as  $B = b/8$  bytes, and the  $u$ -th byte is denoted by  $S_i^{[u]}$ , where  $u \in \{0, 1, \dots, B-1\}$ . If there exists a smallest positive integer  $\ell$  such that every input byte  $S_0^{[u]}$  can influence all output bytes  $S_\ell^{[0]}, S_\ell^{[1]}, \dots, S_\ell^{[B-1]}$  after  $\ell$  iterations  $\pi^\ell$ , then the permutation  $\pi$  is said to achieve full-byte diffusion in  $\ell$  rounds. This minimum number of rounds is denoted by

$$D_{\text{byte}}(\pi) \triangleq \min \left\{ \ell \mid \forall u, v \in \{0, 1, \dots, B-1\}, S_\ell^{[v]} \text{ depends on } S_0^{[u]} \right\}.$$

For the given parameters,  $b = 2560$ , the state consists of twenty 128-bit subblocks, and  $B = b/8 = 320$ . Thus the full-subblock diffusion rounds characterize the diffusion speed

among the twenty subblocks, while the full-byte diffusion rounds characterize fine-grained diffusion among the 320 bytes. These two metrics are jointly used to evaluate the diffusion efficiency of candidate permutation structures in the large-state setting.

Based on the design goals and evaluation metrics above, this work adopts an AES-instruction-oriented large-state permutation framework called PAFF, short for Partial AES Function with Fixed Constant. Its basic idea is to maintain, in every round, two subblock-level paths that can be read in parallel: a local-update path and an original-state coupling path. The local-update path applies one AES round transformation to the 128-bit subblocks selected by  $\Gamma_R$ , while injecting round constants at a small number of positions; the original-state coupling path rearranges the input state of the same round and uses the rearranged subblocks as XOR coupling terms in the output coordinates. As a result, every output subblock contains both a locally nonlinear update result and information from another input subblock, enabling cross-subblock mixing without introducing a heavy full-state linear layer.

This framework naturally matches the execution semantics of AES-instruction platforms. The second input of the `AESENC` instruction is used in the final XOR of the AES round, so a subblock supplied by the original-state coupling path can be placed in that input position to directly realize

$$A(X) \oplus Y,$$

where  $A$  denotes one AES round transformation without an independent round-key XOR, and  $Y$  is a coupling subblock from the input state of the same round. Here  $Y$  is not key material and does not introduce an independent key schedule; rather, it uses the XOR interface already present in AES instructions to realize state-level coupling. Therefore, the second path is referred to in this work as the original-state coupling path, rather than as a feedback branch in the traditional Feistel sense.

Structurally, one round of PAFF can be summarized as selective AES/constant local update, subblock rearrangement on two paths, and coordinate-wise XOR coupling. Correspondingly, the round function is determined by three classes of parameters: the AES/constant selection mask  $\Gamma_R$ , the subblock permutation  $p_1$  on the local-update path, and the subblock permutation  $p_2$  on the original-state coupling path. The mask  $\Gamma_R$  determines which positions use AES-round updates and which positions use round-constant injection;  $p_1$  determines how the local-update results are rearranged into output coordinates; and  $p_2$  determines which subblocks of the original input state enter the corresponding output coordinates as XOR coupling terms. This three-parameter framework keeps the round function regular and easy to implement, while providing a unified interface for tuning cross-subblock diffusion speed, invertibility constraints, and implementation cost.

Formally, let the internal state consist of  $m$  128-bit subblocks, with state space  $\mathcal{S}_m = (\{0, 1\}^{128})^m$ . A state is written as  $S = (S_0, S_1, \dots, S_{m-1})$ , where  $S_j \in \{0, 1\}^{128}$ . Let  $I_m = \{0, 1, \dots, m-1\}$ .

Let  $\text{PAFF}[m]$  denote an iterative permutation over  $\mathcal{S}_m$ . If the permutation consists of  $N_r$  rounds, then

$$\text{PAFF}[m] = R_{N_r-1} \circ R_{N_r-2} \circ \dots \circ R_0,$$

where  $R_i : \mathcal{S}_m \rightarrow \mathcal{S}_m$  denotes the round function in round  $i$ . Given an input state  $S_0 \in \mathcal{S}_m$ , the full permutation is computed as

$$S_{i+1} = R_i(S_i), \quad 0 \leq i < N_r,$$

and outputs  $\text{PAFF}[m](S_0) = S_{N_r}$ .

A single round is determined by three structural parameters:

$$\Gamma_R \in \{0, 1\}^m, \quad p_1 : I_m \rightarrow I_m, \quad p_2 : I_m \rightarrow I_m.$$

Here  $\Gamma_R$  is the AES/constant selection mask,  $p_1$  is the subblock permutation on the local-update path, and  $p_2$  is the subblock permutation on the original-state coupling path.

This specification uses an output-coordinate convention for  $p_1$  and  $p_2$ . That is, the  $j$ -th output subblock takes its inputs from position  $p_1(j)$  on the local-update path and position  $p_2(j)$  on the original-state coupling path.

Let  $A : \{0, 1\}^{128} \rightarrow \{0, 1\}^{128}$  be one AES round transformation without the round-key XOR, i.e., the AES round transformation consisting of SubBytes, ShiftRows, and MixColumns. Let  $RC_i \in \{0, 1\}^{128}$  be the round constant of round  $i$ .

For the round- $i$  input state  $S_i = (S_{i,0}, S_{i,1}, \dots, S_{i,m-1})$ , define the temporary state  $T_i = (T_{i,0}, T_{i,1}, \dots, T_{i,m-1})$  by

$$T_{i,j} = \begin{cases} A(S_{i,j}), & \Gamma_R[j] = 1, \\ RC_i, & \Gamma_R[j] = 0. \end{cases}$$

The round function  $R_i$  is then defined by

$$S_{i+1,j} = R_i(S_i)_j = T_{i,p_1(j)} \oplus S_{i,p_2(j)}, \quad j \in I_m.$$

Figure 3 illustrates the round function in round  $i$ . The input state  $S_i$  is read along two subblock-level paths. The first path is the local-update path: the subblocks specified by  $\Gamma_R$  undergo one AES round transformation, while the constant-injection positions are replaced by the round constant  $RC_i$ ; the resulting temporary state is then rearranged by the subblock permutation  $p_1$ . The second path is the original-state coupling path: the original input state is rearranged by the subblock permutation  $p_2$ . Finally, the two paths are XORed subblock by subblock to obtain the next state  $S_{i+1}$ .

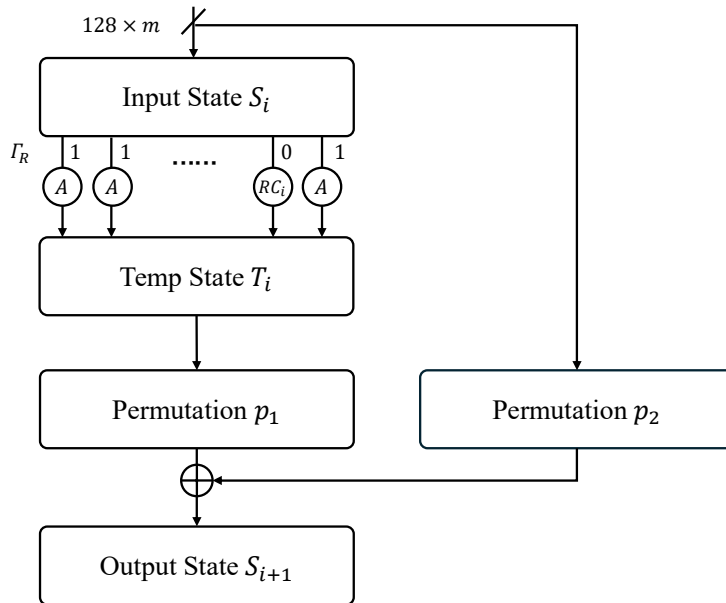


Figure 3: Structure of the round function

On platforms supporting AES instruction sets, when  $\Gamma_R[p_1(j)] = 1$ , the update above can be implemented as

$$S_{i+1,j} = A(S_{i,p_1(j)}) \oplus S_{i,p_2(j)},$$

where the XOR term is supplied as the second input to the **AESENC** instruction. This input is used only to realize the final XOR of the AES round, and no independent key schedule is introduced.

The structural properties of the round function are mainly determined by  $\Gamma_R$ ,  $p_1$ , and  $p_2$ . The mask  $\Gamma_R$  controls the nonlinear-update positions and the constant-injection positions;  $p_1$  determines how the subblocks after the AES round transformation are rearranged in the output; and  $p_2$  determines how original-state subblocks propagate as XOR coupling terms. By jointly selecting  $\Gamma_R$ ,  $p_1$ , and  $p_2$ , one can adjust the cross-subblock diffusion speed and the overall coupling strength while keeping the round function simple to implement.

**Invertibility** From the definition above, the invertibility of the one-round mapping  $R_i$  is not determined by any one of  $\Gamma_R$ ,  $p_1$ , or  $p_2$  alone, but by their combination with the local transformation  $A$ . For convenience, define the relative permutation  $\sigma = p_2 \circ p_1^{-1}$ . This permutation captures the connection relation between the local-update path and the original-state coupling path.

Since  $A : \{0,1\}^{128} \rightarrow \{0,1\}^{128}$  consists of the AES SubBytes, ShiftRows, and MixColumns operations, it is a bijection over  $\{0,1\}^{128}$ . This gives the following sufficient condition for invertibility.

**Theorem 2.3** (A sufficient condition for one-round invertibility). *Let the round- $i$  function  $R_i$  be defined by*

$$S_{i+1,j} = T_{i,p_1(j)} \oplus S_{i,p_2(j)}, \quad j \in I_m,$$

*where  $T_{i,j} = A(S_{i,j})$  if  $\Gamma_R[j] = 1$ , and  $T_{i,j} = RC_i$  if  $\Gamma_R[j] = 0$ . If every cycle of the relative permutation  $\sigma = p_2 \circ p_1^{-1}$  contains at least one position  $j$  satisfying  $\Gamma_R[j] = 0$ , then  $R_i$  is a bijection over the state space  $\mathcal{S}_m$ .*

*Proof.* For any  $k \in I_m$ , let  $j = p_1^{-1}(k)$ . By the definition of the round function,  $S_{i+1,p_1^{-1}(k)} = T_{i,k} \oplus S_{i,\sigma(k)}$ , and hence  $S_{i,\sigma(k)} = S_{i+1,p_1^{-1}(k)} \oplus A(S_{i,k})$  if  $\Gamma_R[k] = 1$ , while  $S_{i,\sigma(k)} = S_{i+1,p_1^{-1}(k)} \oplus RC_i$  if  $\Gamma_R[k] = 0$ .

Consider an arbitrary cycle  $C = (k_0, k_1, \dots, k_{\ell-1})$  of  $\sigma$ , where  $\sigma(k_r) = k_{r+1} \pmod{\ell}$ . If the cycle contains a position  $k_r$  with  $\Gamma_R[k_r] = 0$ , then the equation above directly determines the value of  $S_{i,k_{r+1}}$ . One can then proceed recursively along the cycle: once  $S_{i,k_r}$  is known,  $S_{i,k_{r+1}}$  is uniquely determined. Therefore, starting from this anchor point, all input subblocks on the cycle can be uniquely recovered. Since the cycles of  $\sigma$  are disjoint, the same recovery procedure can be applied independently to every cycle, thereby uniquely determining the entire input state  $S_i$ . Hence  $R_i$  is bijective.  $\square$

The theorem shows that, in parameter selection, it is sufficient to ensure that every cycle of the relative permutation  $\sigma$  contains at least one constant-injection position in order to guarantee one-round invertibility. Consequently, if all rounds satisfy this condition, the full permutation  $\text{PAFF}[m]$  is also bijective.

## 2.5 Parameter Selection

### 2.5.1 Selection of $\text{wt}(\Gamma_R)$

Let the AES/constant selection mask be  $\Gamma_R \in \{0, 1\}^m$ , and denote its Hamming weight by  $\text{wt}(\Gamma_R)$ . This parameter determines the number of subblocks updated by AES-type operations in each round. It therefore simultaneously affects nonlinear injection, cross-subblock diffusion speed, per-round implementation cost, and the total cost required to reach the target number of security rounds. In the permutation framework used here, in order for the round function to satisfy both bijectivity and nonlinearity requirements, the AES/constant selection mask must satisfy

$$1 \leq \text{wt}(\Gamma_R) < m,$$

where  $m$  is the number of subblocks. If  $\text{wt}(\Gamma_R) = 0$ , no AES-type nonlinear update is introduced; if  $\text{wt}(\Gamma_R) = m$ , the structural constraint required for invertibility of the round function is not satisfied. Hence  $\text{wt}(\Gamma_R)$  is not merely an implementation parameter, but a core design parameter that must be jointly balanced among invertibility, nonlinearity, diffusion efficiency, and implementation cost.

Subject to the basic constraints above, this work first screens candidate structures with different values of  $\text{wt}(\Gamma_R)$  according to their diffusion properties. Specifically, for each Hamming weight, it records the best full-subblock diffusion rounds  $\min D_{\text{sub}}$  and the best full-byte diffusion rounds  $\min D_{\text{byte}}$  achieved by the candidate structures. To characterize the rough implementation cost needed to reach full-subblock diffusion, the following simplified metric is further introduced:

$$\text{Cost}(\text{wt}(\Gamma_R)) = \text{wt}(\Gamma_R) \cdot \min D_{\text{sub}}.$$

This metric reflects the tradeoff between the number of AES-type updates per round and the number of rounds required to reach full-subblock diffusion. It is used only for relative comparison in the preliminary diffusion-screening stage and is not equivalent to the true end-to-end implementation cost. The results are shown in Table 4.

Table 4: Diffusion cost under different  $\text{wt}(\Gamma_R)$

| $\text{wt}(\Gamma_R)$ | $\min D_{\text{sub}}$ | $\min D_{\text{byte}}$ | Cost |
|-----------------------|-----------------------|------------------------|------|
| 1                     | —                     | —                      | —    |
| 2                     | —                     | —                      | —    |
| 3                     | 19                    | 20                     | 57   |
| 4                     | 16                    | 16                     | 64   |
| 5                     | 15                    | 16                     | 75   |
| 6                     | 13                    | 14                     | 78   |
| 7                     | 12                    | 13                     | 84   |
| 8                     | 12                    | 12                     | 96   |
| 9                     | 12                    | 12                     | 108  |
| 10                    | 10                    | 11                     | 100  |
| 11                    | 10                    | 10                     | 110  |
| 12                    | 9                     | 10                     | 108  |

Continued on the next page

Table 4: Diffusion cost under different  $\text{wt}(\Gamma_R)$  (continued)

| $\text{wt}(\Gamma_R)$ | $\min D_{\text{sub}}$ | $\min D_{\text{byte}}$ | Cost |
|-----------------------|-----------------------|------------------------|------|
| 13                    | 9                     | 9                      | 117  |
| 14                    | 8                     | 9                      | 112  |
| 15                    | 8                     | 8                      | 120  |
| 16                    | 8                     | 8                      | 128  |
| 17                    | 7                     | 7                      | 119  |
| 18                    | 7                     | 7                      | 126  |
| 19                    | 6                     | 7                      | 114  |

As shown in Table 4, when  $\text{wt}(\Gamma_R)$  is small, no sampled structure satisfying the full-diffusion criterion was found, and the corresponding entries are therefore marked by “–”. As  $\text{wt}(\Gamma_R)$  increases, both the full-subblock and full-byte diffusion rounds decrease overall, indicating that increasing the Hamming weight of the AES/constant selection mask strengthens inter-subblock coupling and shortens the number of rounds required for sufficient diffusion. In particular, when  $\text{wt}(\Gamma_R) = 19$ , the candidate structure reaches

$$\min D_{\text{sub}} = 6, \quad \min D_{\text{byte}} = 7,$$

which reduces the full-subblock diffusion round count compared with  $\text{wt}(\Gamma_R) = 18$ :

$$\min D_{\text{sub}} = 7, \quad \min D_{\text{byte}} = 7.$$

Thus, from the diffusion experiments,  $\text{wt}(\Gamma_R) = 19$  achieves the fastest full-subblock diffusion while satisfying the basic structural constraints.

However, the final choice of  $\text{wt}(\Gamma_R)$  cannot be based solely on diffusion rounds or on the number of AES-type updates in a single round. A smaller  $\text{wt}(\Gamma_R)$  may reduce the number of AES-type updates per round, but it may also substantially increase the number of rounds required to reach the target diffusion strength and security margin. Conversely, a larger  $\text{wt}(\Gamma_R)$  increases the per-round cost but may reduce the overall cost by reducing the total number of rounds. Therefore, the evaluation further benchmarks representative candidate structures and uses the average total number of clock cycles required at the target number of security rounds as the implementation-cost metric.

Specifically, for each value of  $\text{wt}(\Gamma_R)$ , a representative candidate satisfying the diffusion constraints is selected. Under a unified benchmarking framework, the average total cycles and average cycles per byte are measured at the target number of rounds  $R_{\text{sec}} = 2D_{\text{sub}}$ . The results are shown in Fig. 4.

Figure 4 shows that, as  $\text{wt}(\Gamma_R)$  increases, the overall implementation cost required to reach the target number of security rounds generally decreases. This indicates that, in the present structure, reducing the number of AES-type updates per round does not necessarily lead to better performance. When AES-type updates are replaced by XOR updates, the per-round cost decreases, but this saving is not sufficient to offset the additional cost caused by the increased diffusion rounds and target security rounds. In particular, when  $\text{wt}(\Gamma_R) = 19$ , the candidate structure achieves the minimum average total cycles and minimum average cycles per byte, showing that this value provides the best balance among the tested candidates between diffusion speed and overall implementation cost.

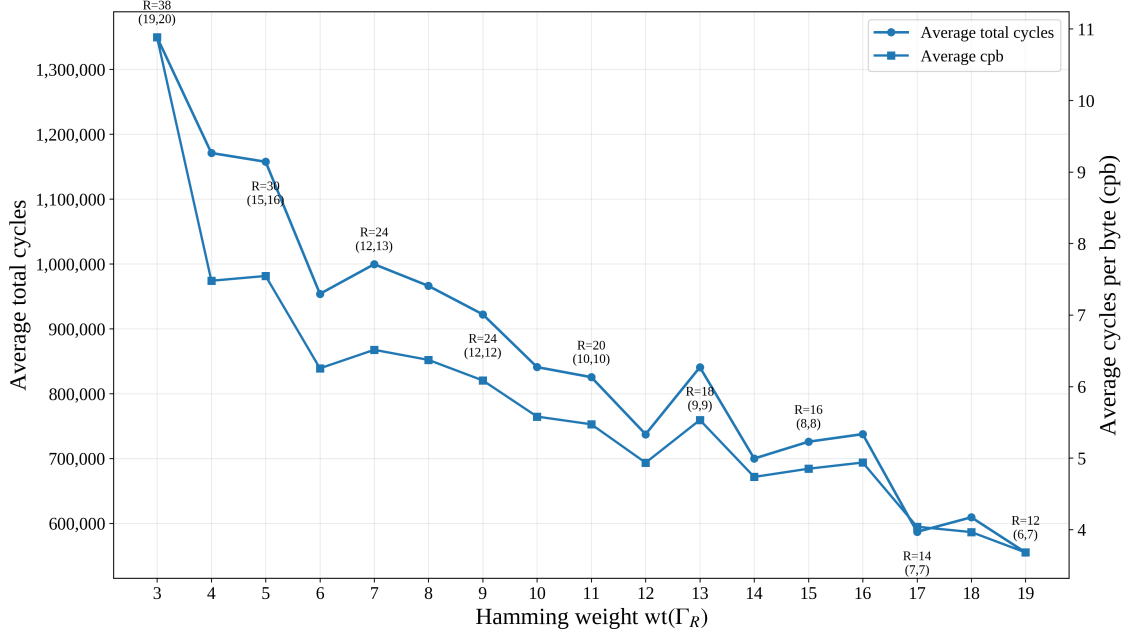


Figure 4: Overall implementation cost for different values of  $wt(\Gamma_R)$ . The left axis gives the average total cycles at the target security rounds  $R_{sec}$ , and the right axis gives the average cycles per byte. The corresponding  $R_{sec}$  and diffusion rounds  $(D_{sub}, D_{byte})$  are also annotated for each candidate.

Taking the bijectivity and nonlinearity constraints, diffusion experiments, and overall cost evaluation into account, this design sets

$$wt(\Gamma_R) = 19$$

in the partial AES layer. This value achieves the fastest full-subblock diffusion while satisfying the structural constraints of the round function, and it gives the lowest overall cost under the security-evaluation and implementation framework used in this work. It is therefore selected as the design parameter of the permutation.

### 2.5.2 Selection of $p_1$ , $p_2$ , and $\Gamma_R$

After fixing the Hamming weight of the AES/constant selection mask  $\Gamma_R$ , it remains to determine the concrete permutation  $p_1$  on the local-update path, the permutation  $p_2$  on the original-state coupling path, and the mask  $\Gamma_R$ . Since the state is divided into twenty 128-bit subblocks, the parameter space has size approximately

$$20! \cdot 20! \cdot \binom{20}{wt(\Gamma_R)},$$

which precludes exhaustive search. This work therefore uses an experimental parameter-selection method based on random sampling and progressive filtering, and determines the final parameters within a unified evaluation framework.

The goal of parameter selection is to identify, within an acceptable computational budget, candidate parameters that simultaneously satisfy bijectivity, fast diffusion, and strong security properties. To reduce sampling bias toward specific structural families,

the search space is first stratified according to the conjugacy classes of the original-state-coupling-path permutation  $p_2$ . In each experiment, a  $p_2$  equivalence class is selected at random, a concrete representative is then randomly generated within that class, and the local-update-path permutation  $p_1$  and the AES/constant selection mask  $\Gamma_R$  satisfying the prescribed Hamming-weight constraint are generated independently at random.

The following progressive filtering procedure is then applied.

**Step 1: Candidate generation** For each trial, a  $p_2$  equivalence class is selected at random, and a concrete original-state-coupling-path permutation  $p_2$  is generated within that class. The local-update-path permutation  $p_1$  and the AES/constant selection mask  $\Gamma_R$  are then generated randomly. The resulting candidate parameter set is denoted by

$$\Pi = (p_1, p_2, \Gamma_R).$$

This step aims to obtain broad coverage of the non-exhaustive candidate space.

**Step 2: Bijectivity filtering** For each candidate parameter set  $\Pi$ , the corresponding linear representation of the round function is first constructed and its bijectivity is tested. Since the underlying transformation in this design must be a permutation, bijectivity is the primary constraint in parameter selection. Candidates that fail this test are discarded and are not evaluated further.

**Step 3: Diffusion filtering** For candidates passing the bijectivity test, the minimum full-subblock diffusion rounds  $D_{\text{sub}}$  and the minimum full-byte diffusion rounds  $D_{\text{byte}}$  are computed. Based on the preceding diffusion analysis, this work preferentially retains candidates satisfying

$$D_{\text{sub}} = 6, \quad D_{\text{byte}} = 7.$$

This step ensures that a disturbance can diffuse to the entire state within a small number of rounds, providing a sufficient diffusion basis for subsequent differential, linear, and integral security analyses.

**Step 4: Differential and linear ranking** For candidates passing the preceding filters, their differential and linear propagation properties are further evaluated. Specifically, the automated models described in the security analysis are used to compute lower bounds on the number of active S-boxes under differential and linear propagation over the target range of rounds. These bounds are used as the main ranking criterion. Candidates with larger lower bounds on the number of active S-boxes in both differential and linear settings are preferred, as they provide a larger margin against differential and linear attacks.

**Step 5: Final selection** When multiple candidates are close in terms of bijectivity, diffusion properties, and lower bounds on active S-boxes, implementation cost is additionally considered. The final permutation parameters are selected from the remaining candidates according to their overall performance.

This unified experimental framework progressively applies stratified sampling, bijectivity filtering, diffusion filtering, differential/linear ranking, and final selection. It balances search coverage, diffusion efficiency, and security margin within a large parameter space,

providing a structured experimental basis for parameter selection. The final permutation parameters are

$$\begin{aligned}\Gamma_R &= (1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1), \\ p_1 &= (19, 4, 6, 17, 9, 0, 13, 11, 8, 5, 15, 10, 16, 3, 12, 14, 18, 7, 2, 1), \\ p_2 &= (1, 0, 3, 4, 5, 2, 7, 8, 9, 10, 6, 12, 13, 14, 15, 16, 17, 18, 19, 11).\end{aligned}$$

### 3 Security of PAVELOR

#### 3.1 Security Claims

Based on the generic security analysis of sponge constructions [8, 9], and in conjunction with the structural security evaluation of the underlying permutation PAFF, the security claims of PAVELOR are divided into claims in the classical model and in the quantum-query model. Unless otherwise stated, the complexities below refer to generic attack complexities; the security of a concrete instance also relies on the absence of structural attacks against the full-round PAFF that improve upon the generic bounds.

##### 3.1.1 Classical Security Claims

In the classical attack model, the target attack complexities of PAVELOR against collision, preimage, and second-preimage attacks are shown in Table 5. Since all three instances use  $c = 2h$ , the sponge capacity does not fall below the classical security target required by the corresponding output length.

Table 5: Security Claims of PAVELOR

| Algorithm    | Collision | Preimage   | Second preimage |
|--------------|-----------|------------|-----------------|
| PAVELOR-512  | $2^{256}$ | $2^{512}$  | $2^{512}$       |
| PAVELOR-768  | $2^{384}$ | $2^{768}$  | $2^{768}$       |
| PAVELOR-1024 | $2^{512}$ | $2^{1024}$ | $2^{1024}$      |

##### 3.1.2 Quantum Security Claims

Sponge-like constructions have been studied in the quantum-query model with respect to collapsing properties, quantum collision resistance, and quantum indistinguishability [13, 2]. This work states quantum security levels under a conservative generic quantum-attack model: for an  $h$ -bit fixed-length output, the generic complexity of quantum collision search is estimated as  $2^{h/3}$ , while the generic complexities of preimage and second-preimage search are estimated according to Grover’s bound, namely  $2^{h/2}$ . On the other hand, the capacity parameters of PAVELOR satisfy  $c = 2h$ , so recovering the hidden capacity part by quantum state recovery has complexity at least  $2^{c/2} = 2^h$ . Therefore, the capacity part does not fall below the output-length-dominated generic quantum security levels shown in Table 6.

Table 6: Quantum Security Claims of PAVELOR

| Algorithm    | Collision    | Preimage  | Second preimage |
|--------------|--------------|-----------|-----------------|
| PAVELOR-512  | $2^{512/3}$  | $2^{256}$ | $2^{256}$       |
| PAVELOR-768  | $2^{256}$    | $2^{384}$ | $2^{384}$       |
| PAVELOR-1024 | $2^{1024/3}$ | $2^{512}$ | $2^{512}$       |

These claims are not an exhaustive proof against all quantum structural attacks. Rather, under the assumption that the underlying permutation has no exploitable structural weakness, they state the generic quantum security levels that PAVELOR can achieve as a fixed-output sponge hash function. In particular, PAVELOR-1024 reaches a  $2^{512}$ -level complexity for quantum preimage and second-preimage search; its quantum collision security is limited by the generic quantum collision bound for fixed-length outputs, namely the  $2^{1024/3}$  level.

## 3.2 Security of the Underlying Permutation

### 3.2.1 Differential and Linear Attacks

Using the byte-level MILP-based automated analysis method for differential and linear properties proposed by Mouha et al. [26], we evaluate the resistance of the underlying permutation against differential and linear attacks. Table 7 gives lower bounds on the number of active S-boxes under differential and linear propagation for different numbers of rounds, where  $AS_D$  and  $AS_L$  denote the number of active S-boxes in the differential and linear settings, respectively.

Table 7: The lower bound for the number of differentially/linearly active S-boxes for PAFF

| Rounds | 1 | 2 | 3 | 4  | 5  | 6  | 7    |
|--------|---|---|---|----|----|----|------|
| $AS_D$ | 0 | 1 | 6 | 19 | 53 | 95 | 125  |
| $AS_L$ | 0 | 1 | 6 | 15 | 30 | 66 | > 86 |

The results above give lower bounds on the numbers of differentially and linearly active S-boxes for 1–7 rounds of the permutation. Combining them with the fact that the maximum differential probability and maximum linear-correlation bound of the AES S-box are both  $2^{-6}$ , the number of active S-boxes obtained by 7 rounds already makes the probability of a single differential or linear characteristic below  $2^{-512}$ . Of course, more refined MILP modeling or characteristic clustering may further improve reduced-round differential or linear distinguishers; nevertheless, given the observed growth rate of active S-boxes and the round margin of the full 24-round permutation, we believe that the full-round permutation provides sufficient resistance to differential and linear analyses.

### 3.2.2 Impossible Differential Attacks

This section evaluates the security of the proposed permutation against impossible differential analysis. Since the work of Biham, Biryukov, and Shamir [10], impossible differential analysis has become an important method for evaluating the security of block ciphers and their internal permutations. Its core idea is to search for input/output differential pairs

whose propagation probability is exactly zero for a given number of rounds. In recent years, constraint-solving and automated-modeling techniques have made it possible to systematically verify specific differential patterns.

In this analysis, both the input difference and the output difference are restricted to a single active byte in order to search for long impossible differential distinguishers. Within this search scope, the search found an **8-round** impossible differential distinguisher; no 9-round single-byte input/output impossible differential distinguisher was found within the tested scope.

The search therefore gives an 8-round single-byte input/output impossible differential distinguisher, and under the same constraints no distinguisher extending to 9 rounds was found. This conclusion does not rule out longer reduced-round impossible differential paths arising from more general truncated-difference patterns, conditions with more active bytes, or other connection techniques. Even if such reduced-round distinguishers can be improved, however, extending them to a distinguisher for the full 24-round permutation would still require bridging a large round gap. We therefore believe that the permutation is secure against this class of attacks.

### 3.2.3 Integral Attacks

The division property of the underlying permutation directly affects integral-type analyses and is therefore an important part of permutation security evaluation. To evaluate resistance to integral analysis, this work uses an automated byte-level division-property search method. The analysis framework is based on Todo’s division-property propagation theory [37, 38] and follows the MILP modeling approach for the division property by Xiang et al. [40]. A byte-level model adapted to the proposed permutation is then constructed. In the search, one byte of the input state is fixed as a constant, all remaining bytes are active, and the model checks whether balanced output bytes still exist after a given number of rounds.

Under the above input pattern and modeling framework, the search found integral distinguishers up to **8 rounds**. In particular, a division property still exists after 8 rounds, as shown in Table 8, while no exploitable 9-round integral distinguisher was observed in this search setting.

These experiments show that, in the byte-level division-property model used in this work, integral properties can cover up to 8 rounds, while no exploitable balanced output byte was observed at 9 rounds. Combining more fine-grained bit-level division properties with different input-set constructions may lead to stronger reduced-round integral analyses. Even so, the current integral distinguisher remains far from the full 24 rounds, and we believe that the full-round permutation is not affected by practical integral attacks.

### 3.2.4 Exchange-Equivalence Attacks

The exchange-equivalence attack [6] is a differential analysis technique for SPN-like block ciphers under chosen plaintexts. It distinguishes a block cipher from a random permutation by studying the exchange property of particular plaintext differences and the corresponding zero-difference patterns in the ciphertexts. For the proposed permutation, the analysis constructs 6-round and 7-round exchange-equivalence distinguishers. The basic idea is to select two plaintext pairs with an exchange relation at the input and to examine whether

Table 8: Distribution of output-byte division properties ( $B$  denotes a balanced byte and  $U$  denotes an unbalanced byte)

| Branch/byte | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
|-------------|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|
| B00         | B | B | B | B | B | B | B | B | B | B | B  | B  | B  | B  | B  | B  |
| B01         | B | B | B | B | B | B | B | B | B | B | B  | B  | B  | B  | B  | B  |
| B02         | B | B | B | B | B | B | B | B | B | B | B  | B  | B  | B  | B  | B  |
| B03         | B | B | B | B | B | B | B | B | B | B | B  | B  | B  | B  | B  | B  |
| B04         | U | U | U | U | U | U | U | U | U | U | U  | U  | U  | U  | U  | U  |
| B05         | U | U | U | U | B | U | U | U | U | U | U  | U  | U  | U  | U  | U  |
| B06         | B | B | B | B | B | B | B | B | B | B | B  | B  | B  | B  | B  | B  |
| B07         | B | B | B | B | B | B | B | B | B | B | B  | B  | B  | B  | B  | B  |
| B08         | B | B | B | U | B | B | U | U | B | B | B  | B  | U  | U  | U  | U  |
| B09         | B | B | B | B | B | B | B | B | B | B | B  | B  | B  | B  | B  | B  |
| B10         | B | B | B | B | B | B | B | B | B | B | B  | B  | B  | B  | B  | B  |
| B11         | B | B | B | B | B | B | B | B | B | B | B  | B  | B  | B  | B  | B  |
| B12         | B | B | B | B | B | B | B | B | B | B | B  | B  | B  | B  | B  | B  |
| B13         | B | B | B | B | B | B | B | B | B | B | B  | B  | B  | B  | B  | B  |
| B14         | B | B | B | B | B | B | B | B | B | B | B  | B  | B  | B  | B  | B  |
| B15         | B | B | B | B | B | B | B | B | B | B | B  | B  | B  | B  | B  | B  |
| B16         | B | B | B | B | B | B | B | B | B | B | B  | B  | B  | B  | B  | B  |
| B17         | B | B | B | B | B | B | B | B | B | B | B  | B  | B  | B  | B  | B  |
| B18         | B | B | B | B | B | B | B | B | B | B | B  | B  | B  | B  | B  | B  |
| B19         | B | B | B | B | B | B | B | B | B | B | B  | B  | B  | B  | B  | B  |

deterministic relations with probability one appear at specific output positions after encryption.

The 6-round distinguisher is constructed as follows. First, construct a plaintext pair  $(P_0, P_1)$  such that the plaintexts have a nonzero  $m$ -main-diagonal difference at the third subblock, where  $2 \leq m < 4$ , and all other main-diagonal and subblock differences are zero. Then construct a second plaintext pair  $(P'_0, P'_1)$  by exchanging one nonzero-difference main diagonal of  $(P_0, P_1)$ . After applying 6 rounds to both plaintext pairs, the distinguisher succeeds if

$$R^6(P_0) \oplus R^6(P_1) = R^6(P'_0) \oplus R^6(P'_1)$$

holds with probability one at subblocks 7 and 17. For a random permutation, the probability of this event is  $2^{-256}$ . As shown in Fig. 5, gray cells indicate nonzero differences in the corresponding subblocks of the plaintext pairs, white cells indicate zero differences, and red boxes indicate positions where the two plaintext pairs have equal differences.

Based on the 6-round distinguisher, a 7-round distinguisher is constructed in which the seventh round contains only the  $A$  transformation. After 7 rounds are applied to the two plaintext pairs with an exchange relation, the corresponding two ciphertext pairs have the same zero-difference pattern at subblocks 7 and 17 with probability one. For a random permutation, the probability of this event is  $2^{-8}$ .

The exchange-equivalence analysis reveals low-round structural relations that still exist in the 6- and 7-round permutation. Such relations help explain the diffusion process in the early rounds, but their constrained positions and probability forms do not directly yield collision or preimage attacks on the full hash instances. More sophisticated analysis techniques may in the future produce exchange-type distinguishers covering more rounds; nevertheless, for the full 24-round permutation, we believe that these low-round relations are sufficiently destroyed by the subsequent diffusion and nonlinear layers.

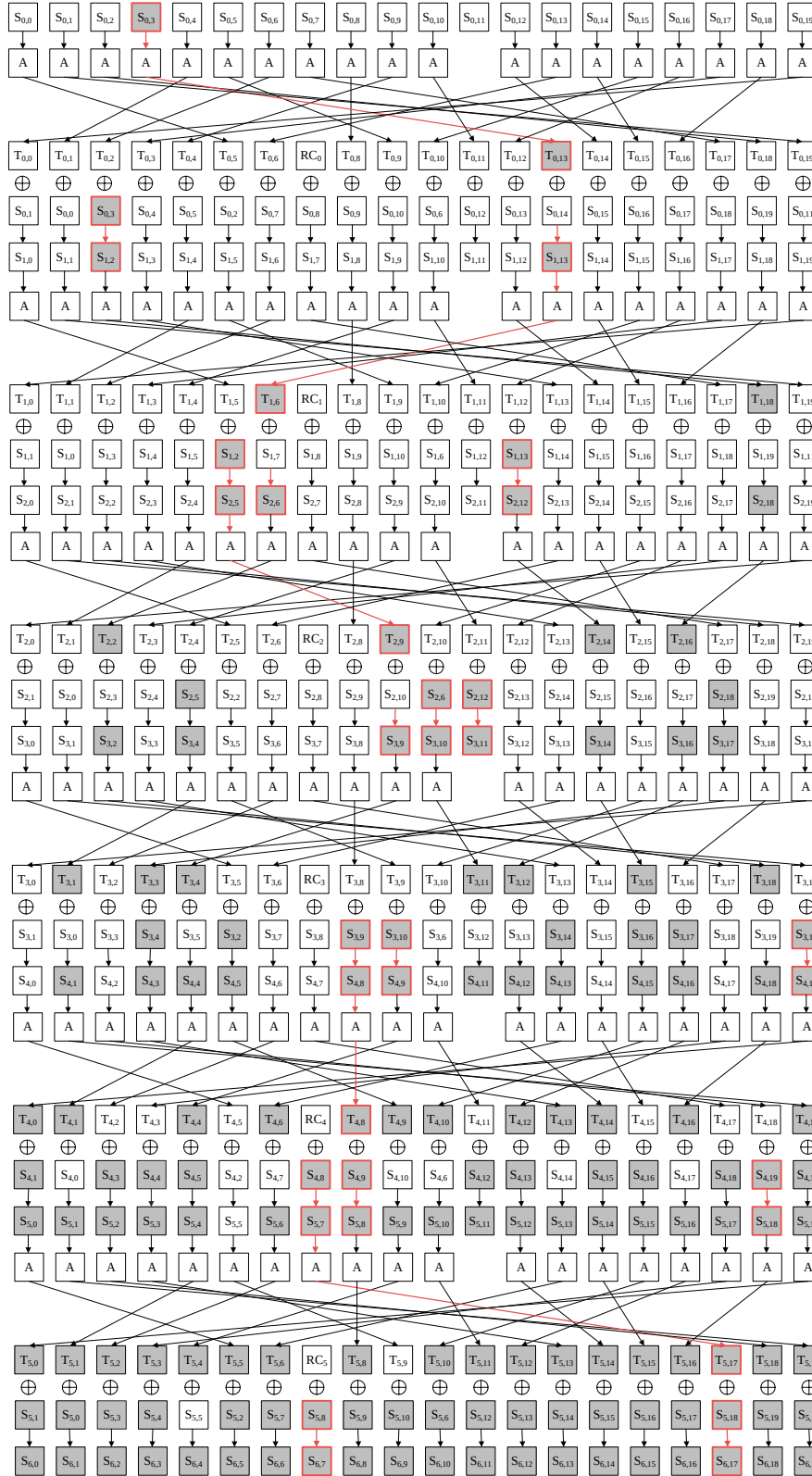


Figure 5: 6-round exchange-equivalence distinguisher

### 3.3 Security of the Hash Algorithm

#### 3.3.1 Resistance to Length-Extension Attacks

Length-extension attacks primarily target Merkle–Damgård-type iterative hash constructions. In such constructions, the final digest is usually equal to, or can be directly derived from, the last chaining value. An attacker who knows  $\text{Hash}(M)$  and the message length  $|M|$  can then use the digest as a new initial chaining value and continue processing appended message blocks, thereby computing the digest of an extended message of the form

$$\text{Hash}(M \parallel \text{pad}(M) \parallel X).$$

The essence of the attack is that the digest leaks enough internal state to continue the compression-function iteration.

PAVELOR uses a sponge construction and does not output the full internal state. Let the internal state after padding and absorbing a message  $M$  be

$$S_M = (S_{\text{rate}}, S_{\text{cap}}) \in \{0, 1\}^{r+c},$$

where  $|S_{\text{rate}}| = r$  and  $|S_{\text{cap}}| = c$ . The digest-generation phase starts from  $S_M$  and outputs only  $h$  bits through the squeezing process:

$$Y = \text{PAVELOR-}h(M) = \text{Trunc}_h(\text{Squeeze}(S_M, h, r)).$$

Even if an attacker knows  $Y$  and  $|M|$ ,  $Y$  does not determine the full state  $S_M$ . In particular, the capacity part  $S_{\text{cap}}$  remains hidden throughout absorbing and squeezing. To continue absorbing appended data, the attacker would have to recover the full state, or construct an internal state equivalent to  $S_M$  for all subsequent absorbing and squeezing operations. From the viewpoint of generic attacks, this requires determining the hidden capacity part, with classical complexity at least  $2^c$  and Grover-search quantum complexity at least  $2^{c/2}$ . Since all three instances in this work satisfy  $c = 2h$ , this state-recovery cost is not lower than the corresponding preimage-security target implied by the output length.

Moreover, PAVELOR uses the prefix-unambiguous padding rule

$$\text{Pad}(M) = M \parallel 1 \parallel 0^k \parallel 1,$$

which deterministically encodes message boundaries and termination positions for messages of different lengths. The padding rule ensures unambiguous message encoding, while resistance to length-extension attacks relies on the fact that the PAVELOR digest is not an internal chaining value and that the capacity part is not exposed. Therefore, even if the original message length is known, an attacker cannot compute the digest of an extended message solely from  $\text{PAVELOR-}h(M)$ ; structurally, PAVELOR avoids the typical length-extension weakness of Merkle–Damgård hash functions.

#### 3.3.2 Meet-in-the-Middle Attacks

Meet-in-the-middle attacks are effective techniques for constructing preimage and collision attacks on hash functions. Their analytical frameworks and implementation techniques have been developed over a long period and, in recent years, have been combined

with automated methods and widely applied to hash-function design and security evaluation [3, 22, 17, 5, 31, 18]. At EUROCRYPT 2021, Bao et al. proposed a meet-in-the-middle preimage-attack framework for AES-like hash functions [5] and modeled characteristic propagation using MILP. At EUROCRYPT 2023, Qin et al. first proposed a generic meet-in-the-middle attack framework for sponge-based hash functions [31]. Dong et al. subsequently generalized the framework at CRYPTO 2024 so that it covers meet-in-the-middle preimage and collision attacks on a wider range of sponge-based hash functions [18].

**Preimage attacks** For preimage attacks, this work first considers the improved meet-in-the-middle framework of Dong et al. [18]. That framework can circumvent the  $2^{h/2}$  lower-bound limitation for sponge preimage attacks in the framework of Qin et al. when  $c/2 < h$ . However, the instances of PAVELOR satisfy  $c = 2h$ , i.e.,  $c/2 = h$ , under which the improved framework is more complex to analyze and does not lead to a better estimate in the present setting. The analysis therefore adopts the meet-in-the-middle preimage-attack framework of Qin et al. [31].

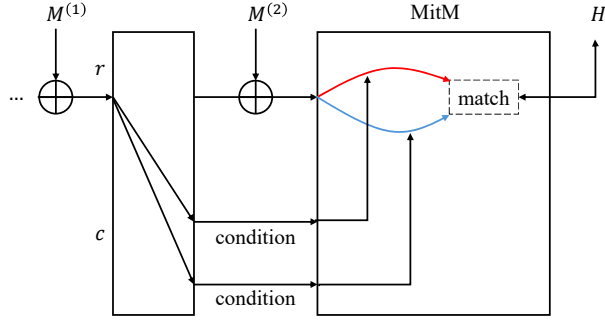


Figure 6: Meet-in-the-middle preimage-attack framework for sponge constructions [31]

Using the meet-in-the-middle preimage model of Qin et al., the MILP-based automated search gives the 7-round MitM path shown in Fig. 7. In the figure, each  $4 \times 4$  small matrix represents a 128-bit substate, with cells arranged according to the column-major order of the AES state matrix. The colors denote different dependency types in the MitM search: red cells depend only on red-side neutral words, blue cells depend only on blue-side neutral words, green cells are linear combinations of red and blue neutral words, gray cells are constants, and white cells are nonlinear combinations of red and blue neutral words. In the final layer, substates enclosed by dashed boxes form the digest-output region. The values following the red and blue squares on the right-hand side indicate the consumption of red-side and blue-side neutral-word degrees of freedom in the corresponding round.

Let the remaining red and blue degrees of freedom at the matching layer be  $d_R$  and  $d_B$  bits, respectively, and let the number of matched bits be  $t$ . Let the output length be  $h$ . In one complete forward/backward meet-in-the-middle search, i.e., one MitM episode,  $2^{d_R}$  red-side configurations and  $2^{d_B}$  blue-side configurations are enumerated. Each pair passes the  $t$ -bit matching condition with probability approximately  $2^{-t}$ . Hence the expected number of partial target preimages produced by one episode is

$$N_{\text{pt}} \approx 2^{d_R + d_B - t}.$$

Since the matching layer fixes only  $t$  output bits, the remaining  $h - t$  output bits must still match the given target digest. Therefore, the number of MitM episodes required to obtain

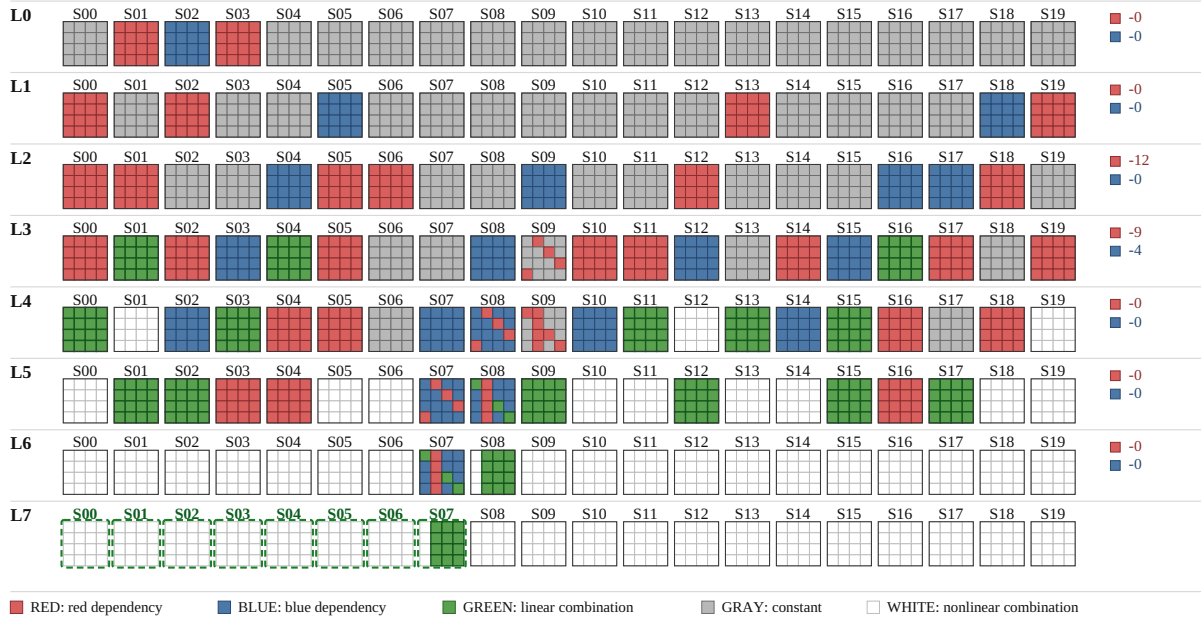


Figure 7: 7-round meet-in-the-middle characteristic

one complete preimage is approximately

$$N_{\text{ep}} \approx \frac{2^{h-t}}{N_{\text{pt}}} = 2^{h-d_R-d_B}.$$

The time of one MitM episode is dominated by the enumeration cost on the red and blue sides and can be approximated by  $T_{\text{ep}} \approx 2^{\max(d_R, d_B)}$ , while the memory required for the lookup table is  $M \approx 2^{\min(d_R, d_B)}$ . Thus the total time complexity of the preimage attack is

$$\begin{aligned} T_{\text{total}} &\approx N_{\text{ep}} \cdot T_{\text{ep}} \\ &\approx 2^{h-d_R-d_B} \cdot 2^{\max(d_R, d_B)} \\ &= 2^{h-\min(d_R, d_B)}. \end{aligned}$$

For the 7-round MitM configuration found by the search, the red and blue degrees of freedom and matching bits at the meeting layer are

$$d_R = 11 \times 8 = 88, \quad d_B = 12 \times 8 = 96, \quad t = 12 \times 8 = 96,$$

where these values are derived from the 11 remaining red bytes, 12 remaining blue bytes, and 12 matching bytes at the meeting layer. For output length  $h = 1024$ , substituting into the formula gives

$$T_{\text{total}} \approx 2^{1024-\min(88,96)} = 2^{936}$$

7-round permutation calls. The memory complexity is approximately

$$M \approx 2^{\min(88,96)} = 2^{88}$$

intermediate states. The resulting 7-round meet-in-the-middle preimage-attack structure has theoretical time complexity about  $2^{936}$  7-round permutation calls for PAVELOR-1024. This result shows that the reduced-round permutation contains local structure that can

be exploited by red and blue degrees of freedom. Further optimizing the matching layer, freedom allocation, or neutral-word selection may reduce the complexity of reduced-round preimage attacks. However, the attack currently covers only 7 rounds, remains far from the full 24 rounds, and has complexity well beyond practical feasibility. We therefore believe that full-round PAVELOR resists meet-in-the-middle preimage attacks.

**Collision attacks** For collision attacks, Dong et al. [18] describe two meet-in-the-middle collision-attack models for sponge constructions, shown on the left and right as Collision Framework I and Collision Framework II, respectively. Since the algorithm instances in this work satisfy  $c = 2h$ , Collision Framework I is more suitable for the present parameter setting.

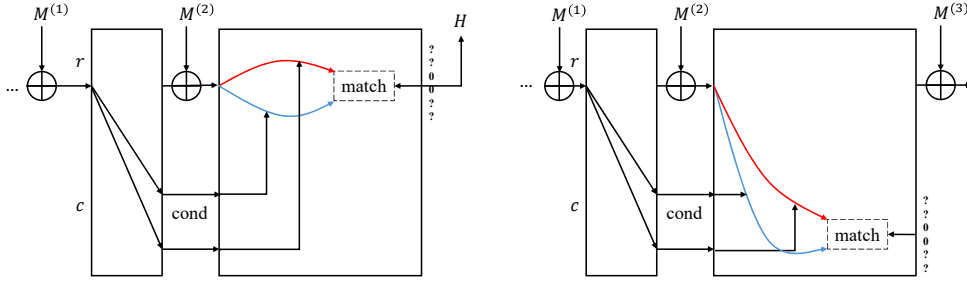


Figure 8: Meet-in-the-middle collision-attack framework for sponge constructions [18]

Based on the automatically found 7-round meet-in-the-middle characteristic, a corresponding target-collision attack can be constructed under Collision Framework I. The path contains 32 red bytes and 16 blue bytes in the initial state. After 7 rounds of propagation,  $t = 12$  matching bytes remain at the matching layer, while the remaining effective degrees of freedom on the two sides are  $d_R = 11$  and  $d_B = 12$  bytes. The estimate here uses bytes as the basic unit, and the 1024-bit output corresponds to  $n_b = 128$  output bytes.

In Collision Framework I, the first step fixes  $t$  output bytes and uses the meet-in-the-middle path to find partial target preimages satisfying this fixed prefix. According to the generic model, the unit cost can be estimated as

$$C_I \approx 2^{8(t - \min(d_R, d_B, t))}.$$

For this configuration,  $\min(d_R, d_B, t) = \min(11, 12, 12) = 11$ , and therefore

$$C_I \approx 2^{8(12-11)} = 2^8.$$

To obtain a final collision from the partial target preimages, the required number of partial target preimages is approximately

$$2^{8(n_b - t)/2} = 2^{8(128-12)/2} = 2^{464}.$$

Combining the meet-in-the-middle stage with the subsequent birthday-merging stage gives

the total time complexity of this 7-round collision attack:

$$\begin{aligned}
\text{Time}_{\text{coll}} &\approx C_I \cdot 2^{8(n_b-t)/2} \\
&= 2^{8(t-\min(d_R, d_B, t))} \cdot 2^{8(n_b-t)/2} \\
&= 2^{8(\frac{n_b+t}{2}-\min(d_R, d_B, t))} \\
&= 2^{8(\frac{128+12}{2}-11)} \\
&= 2^{472}.
\end{aligned}$$

Thus, under Collision Framework I, one obtains a 7-round meet-in-the-middle collision-attack structure for PAVELOR-1024, with time and space complexities approximately  $2^{472}$  and  $2^{88}$ , respectively. This complexity is below the birthday bound for a 1024-bit output, indicating that the reduced-round structure still leaves collision-path space worthy of study. Future improvements in partial target-preimage generation or neutral-word allocation may yield better reduced-round collision attacks. Nevertheless, we believe that the full 24-round permutation provides sufficient accumulation of diffusion and nonlinearity, making such meet-in-the-middle collision attacks difficult to extend to the full-round instances.

### 3.3.3 Rebound Attacks

For AES-like hash designs, rebound attacks typically follow a start-from-the-middle strategy and divide the attack into an inbound phase and an outbound phase. Following the analysis method of Areion [24], the classical two-round Super-Sbox can absorb 38 active S-boxes, while a three-round variant can cover at most 54 active S-boxes but requires each local unit to pay an additional success-probability cost of  $2^{-64}$ . Moreover, under an attacker-favorable estimate, the outbound phase only pays a byte-level cancellation cost for half of the remaining active S-boxes, so its probability can be upper bounded by  $2^{-8\lceil A/2 \rceil}$ .

The three sponge instances of PAVELOR share the same 2560-bit underlying permutation, with parameters

$$(r, c) = (1536, 1024), (1024, 1536), (512, 2048),$$

corresponding to output lengths of 512, 768, and 1024 bits. For these instances, the numbers of 128-bit branches in the rate part are

$$B_r = \frac{r}{128} = 12, 8, 4.$$

Here  $B_r$  is the number of local branches directly supported by rate-side degrees of freedom on the middle cut. Since the round function contains AES-like nonlinear branches, XOR branches, and subblock permutations, a local inbound unit that can be solved independently and extended in both directions cannot occupy only one branch. Conservatively, a two-round Super-Sbox local unit occupies at least three middle-cut branches, while a three-round variant occupies at least four middle-cut branches. Hence the numbers of independently deployable local units satisfy

$$N_2 \leq \left\lfloor \frac{B_r}{3} \right\rfloor, \quad N_3 \leq \left\lfloor \frac{B_r}{4} \right\rfloor,$$

namely

$$N_2 = (4, 2, 1), \quad N_3 = (3, 2, 1),$$

for the 512-, 768-, and 1024-bit output instances, respectively.

Let  $AS_D(R)$  denote the lower bound on the number of differential active S-boxes for  $R$  rounds of the permutation, and let  $[x]_+ = \max\{x, 0\}$ . For the two-round Super-Sbox, we have

$$p_{\text{reb}}^{(2)}(R) \leq 2^{-8 \lceil [AS_D(R) - 38N_2]_+ / 2 \rceil}.$$

To ensure that the attack is no better than the birthday bound  $2^{-h/2}$  for an  $h$ -bit-output hash function, it suffices to require

$$8 \left\lceil \frac{[AS_D(R) - 38N_2]_+}{2} \right\rceil \geq \frac{h}{2}.$$

This gives the required thresholds for the total number of active S-boxes as

$$AS_D(R) \geq 215, 171, 165,$$

for the 512-, 768-, and 1024-bit output instances, respectively.

For the three-round Super-Sbox variant,

$$p_{\text{reb}}^{(3)}(R) \leq 2^{-64N_3} \cdot 2^{-8 \lceil [AS_D(R) - 54N_3]_+ / 2 \rceil}.$$

It therefore suffices to require

$$64N_3 + 8 \left\lceil \frac{[AS_D(R) - 54N_3]_+}{2} \right\rceil \geq \frac{h}{2}$$

to ensure that the attack complexity is no better than the birthday bound. The resulting required thresholds for the total number of active S-boxes are

$$AS_D(R) \geq 177, 171, 165,$$

again for the 512-, 768-, and 1024-bit output instances, respectively.

On the other hand, from the differential-attack analysis we have

$$AS_D(1 \sim 7) = 0, 1, 6, 19, 53, 95, 125,$$

and lower bounds on the number of differential active S-boxes can be added over rounds. Thus it follows directly that

$$AS_D(12) \geq AS_D(7) + AS_D(5) = 125 + 53 = 178,$$

$$AS_D(13) \geq AS_D(7) + AS_D(6) = 125 + 95 = 220,$$

$$AS_D(24) \geq AS_D(7) + AS_D(7) + AS_D(7) + AS_D(3) = 381.$$

Comparing these lower bounds with the thresholds gives the following conclusions:

- Under the two-round Super-Sbox model, the active-S-box thresholds for the 512-, 768-, and 1024-bit output instances are 215, 171, and 165, respectively. Thus the 512-bit instance reaches the threshold corresponding to  $2^{256}$  collision complexity from 13 rounds onward, while the 768- and 1024-bit instances reach the thresholds corresponding to  $2^{384}$  and  $2^{512}$  collision complexity from 12 rounds onward.

- Under the three-round Super-Sbox model, the required thresholds for the 512-, 768-, and 1024-bit output instances are 177, 171, and 165, respectively. Hence all three instances reach the active-S-box thresholds corresponding to their collision complexities from 12 rounds onward.

In the rebound estimates above, the three output-length instances of PAVELOR reach the collision-complexity thresholds of the corresponding models within 12–13 rounds, while the active-S-box lower bound for the full 24-round permutation,  $AS_D(24) \geq 381$ , is clearly above these thresholds. More refined inbound/outbound divisions, local Super-Sbox solving strategies, or freedom-matching techniques may improve reduced-round rebound-type collision paths. In the full-round setting, however, an attacker would have to overcome both a high active-S-box cost and the limited degrees of freedom available from the rate part. Based on the current analysis, we believe that full-round PAVELOR retains a security margin against rebound-type collision attacks and the preimage attacks derived from them.

## 4 Performance of PAVELOR

This section evaluates the software efficiency of the optimized PAVELOR implementation on general-purpose x86 platforms. The tests focus on two aspects: first, a same-platform comparison with common hash algorithms for several representative message lengths; second, measurements for the S1–S8 input-length classes specified by the New-Generation Commercial Cryptographic Algorithm x86-Architecture Implementation Self-Evaluation Guidelines. Unless otherwise stated, all tests are single-threaded and report average latency, average cycle count, cycles per byte (cpb), and message throughput. Here cpb reflects the instruction-level cost of the implementation, while throughput reflects end-to-end processing capability on the concrete test platform.

### 4.1 x86 Platform Performance Tests

This subsection tests the optimized PAVELOR implementation on the S1–S8 input lengths specified by the self-evaluation guidelines and gives comparison results for several public hash implementations under the same benchmark program and input-generation rules. The measurements cover only the optimized implementation, not the reference implementation.

The test platform is a dual-socket AMD EPYC 9754 processor system running Ubuntu 24.04.1 LTS, with GCC/G++ 13.3.0 as the compiler. SHA3, SM3, and BLAKE2 are provided by OpenSSL 3.0.13; BLAKE3 uses the official C implementation and is restricted to code paths no higher than AVX2; Areion-MD uses the public AVX2 implementation. The PAVELOR source files are compiled as C99, and the benchmark program is compiled as C++11. The main optimization options are

```
-O3 -march=x86-64 -mavx2 -maes -mtune=native -flto -fomit-frame-pointer
```

The benchmark process is pinned to a single logical core. The input lengths follow the self-evaluation guidelines:

| Level    | S1 | S2  | S3  | S4   | S5   | S6   | S7    | S8    |
|----------|----|-----|-----|------|------|------|-------|-------|
| Length/B | 32 | 128 | 512 | 1024 | 4096 | 8192 | 16384 | 65536 |

Each input length is measured 1000 times. Test messages are generated deterministically, and the reported metrics are average latency, average cycle count, cpb, and MiB/s throughput.

Table 9 gives the S1–S8 results for the three PAVELOR output-length instances on this platform. For the long-message steady-state S8 input, PAVELOR-512, PAVELOR-768, and PAVELOR-1024 reach 1580.12 MiB/s, 1193.55 MiB/s, and 622.55 MiB/s, respectively; the corresponding cpb values are 1.351, 1.791, and 3.440.

Table 9: S1–S8 test results of the optimized PAVELOR implementation on the x86 platform

| Alg.         | Lvl. | In/B  | Out/B | Avg. ns  | Avg. cyc. | cpb    | MiB/s   |
|--------------|------|-------|-------|----------|-----------|--------|---------|
| PAVELOR-512  | S1   | 32    | 64    | 290.5    | 296.7     | 9.272  | 105.04  |
|              | S2   | 128   | 64    | 276.7    | 299.0     | 2.336  | 441.13  |
|              | S3   | 512   | 64    | 505.2    | 804.3     | 1.571  | 966.52  |
|              | S4   | 1024  | 64    | 855.3    | 1580.9    | 1.544  | 1141.82 |
|              | S5   | 4096  | 64    | 2712.2   | 5719.1    | 1.396  | 1440.23 |
|              | S6   | 8192  | 64    | 5143.9   | 11163.0   | 1.363  | 1518.80 |
|              | S7   | 16384 | 64    | 10110.3  | 22311.4   | 1.362  | 1545.45 |
|              | S8   | 65536 | 64    | 39553.9  | 88541.5   | 1.351  | 1580.12 |
| PAVELOR-768  | S1   | 32    | 96    | 289.6    | 304.3     | 9.510  | 105.37  |
|              | S2   | 128   | 96    | 372.7    | 495.7     | 3.873  | 327.52  |
|              | S3   | 512   | 96    | 678.1    | 1177.1    | 2.299  | 720.08  |
|              | S4   | 1024  | 96    | 1091.7   | 2106.9    | 2.057  | 894.57  |
|              | S5   | 4096  | 96    | 3534.9   | 7587.9    | 1.853  | 1105.05 |
|              | S6   | 8192  | 96    | 6829.2   | 14973.7   | 1.828  | 1143.98 |
|              | S7   | 16384 | 96    | 13300.2  | 29509.6   | 1.801  | 1174.79 |
|              | S8   | 65536 | 96    | 52365.0  | 117376.4  | 1.791  | 1193.55 |
| PAVELOR-1024 | S1   | 32    | 128   | 397.8    | 522.4     | 16.326 | 76.73   |
|              | S2   | 128   | 128   | 596.8    | 976.9     | 7.632  | 204.56  |
|              | S3   | 512   | 128   | 1182.2   | 2290.7    | 4.474  | 413.03  |
|              | S4   | 1024  | 128   | 1952.0   | 4002.9    | 3.909  | 500.29  |
|              | S5   | 4096  | 128   | 6666.5   | 14618.0   | 3.569  | 585.96  |
|              | S6   | 8192  | 128   | 12865.2  | 28550.3   | 3.485  | 607.26  |
|              | S7   | 16384 | 128   | 25334.1  | 56561.2   | 3.452  | 616.76  |
|              | S8   | 65536 | 128   | 100393.6 | 225433.7  | 3.440  | 622.55  |

Table 10 gives the comparison results for hash algorithms measured on the same platform with the same benchmark program and input-generation rules. For the S8 long-message input, SHA3-512 reaches 219.12 MiB/s, SM3 reaches 304.74 MiB/s, BLAKE2b-512 reaches 758.78 MiB/s, BLAKE3-256 reaches 1740.00 MiB/s on code paths no higher than AVX2, and Areion-MD reaches 902.72 MiB/s. Therefore, the S8 throughput of PAVELOR-512 is approximately 7.21 times that of SHA3-512, 5.18 times that of SM3, 2.08 times that of BLAKE2b-512, and 1.75 times that of Areion-MD. BLAKE3-256 still shows high throughput in the long-message tree-hashing setting, about 1.10 times that of PAVELOR-512.

BLAKE3, a representative member of the BLAKE family, is among the highest-throughput general-purpose hash functions in modern software implementations; its specification explicitly targets high speed across platforms, SIMD parallelism, and thread-level parallelism [30]. It therefore provides an important performance baseline for large-scale data-integrity verification. In large-model training and deployment pipelines, dataset shards, model weights, checkpoint files, and data blocks transferred across distributed systems all require high-throughput integrity checks; in this long-message setting, the S8

throughput of PAVELOR-512 reaches about 90.8% of BLAKE3-256 and about 2.08 times that of BLAKE2b-512.

Table 10: S1–S8 test results of comparison hash algorithms on the x86 platform

| Alg.        | Lvl. | In/B  | Out/B | Avg. ns  | Avg. cyc. | cpb    | MiB/s   |
|-------------|------|-------|-------|----------|-----------|--------|---------|
| SHA3-256    | S1   | 32    | 32    | 827.7    | 1539.8    | 48.118 | 36.87   |
|             | S2   | 128   | 32    | 818.0    | 1524.9    | 11.914 | 149.23  |
|             | S3   | 512   | 32    | 1754.3   | 3628.7    | 7.087  | 278.33  |
|             | S4   | 1024  | 32    | 3022.0   | 6471.7    | 6.320  | 323.16  |
|             | S5   | 4096  | 32    | 10291.2  | 22804.4   | 5.567  | 379.57  |
|             | S6   | 8192  | 32    | 19793.1  | 44182.4   | 5.393  | 394.71  |
|             | S7   | 16384 | 32    | 38885.0  | 87127.4   | 5.318  | 401.83  |
|             | S8   | 65536 | 32    | 152825.4 | 343438.8  | 5.240  | 408.96  |
| SHA3-512    | S1   | 32    | 64    | 846.6    | 1549.7    | 48.429 | 36.05   |
|             | S2   | 128   | 64    | 1161.0   | 2254.4    | 17.613 | 105.14  |
|             | S3   | 512   | 64    | 3021.2   | 6459.2    | 12.616 | 161.62  |
|             | S4   | 1024  | 64    | 5223.1   | 11407.5   | 11.140 | 186.97  |
|             | S5   | 4096  | 64    | 18369.0  | 40954.5   | 9.999  | 212.66  |
|             | S6   | 8192  | 64    | 36128.3  | 80908.2   | 9.876  | 216.24  |
|             | S7   | 16384 | 64    | 71982.0  | 161579.6  | 9.862  | 217.07  |
|             | S8   | 65536 | 64    | 285233.6 | 641342.9  | 9.786  | 219.12  |
| SM3         | S1   | 32    | 32    | 706.7    | 1253.8    | 39.181 | 43.18   |
|             | S2   | 128   | 32    | 1218.5   | 2425.5    | 18.949 | 100.18  |
|             | S3   | 512   | 32    | 2361.0   | 4991.9    | 9.750  | 206.81  |
|             | S4   | 1024  | 32    | 3968.4   | 8605.1    | 8.403  | 246.09  |
|             | S5   | 4096  | 32    | 13539.5  | 30109.8   | 7.351  | 288.51  |
|             | S6   | 8192  | 32    | 26270.9  | 58762.0   | 7.173  | 297.38  |
|             | S7   | 16384 | 32    | 51831.6  | 116268.6  | 7.096  | 301.46  |
|             | S8   | 65536 | 32    | 205092.8 | 461038.3  | 7.035  | 304.74  |
| BLAKE2s-256 | S1   | 32    | 32    | 628.1    | 1079.1    | 33.722 | 48.59   |
|             | S2   | 128   | 32    | 768.0    | 1412.3    | 11.034 | 158.94  |
|             | S3   | 512   | 32    | 1593.8   | 3269.6    | 6.386  | 306.37  |
|             | S4   | 1024  | 32    | 2677.1   | 5703.6    | 5.570  | 364.79  |
|             | S5   | 4096  | 32    | 9126.3   | 20197.5   | 4.931  | 428.02  |
|             | S6   | 8192  | 32    | 17803.8  | 39712.3   | 4.848  | 438.81  |
|             | S7   | 16384 | 32    | 34828.3  | 78002.2   | 4.761  | 448.63  |
|             | S8   | 65536 | 32    | 137483.3 | 308924.4  | 4.714  | 454.60  |
| BLAKE2b-512 | S1   | 32    | 64    | 684.2    | 1186.6    | 37.082 | 44.60   |
|             | S2   | 128   | 64    | 671.6    | 1177.7    | 9.201  | 181.76  |
|             | S3   | 512   | 64    | 1217.2   | 2398.9    | 4.685  | 401.15  |
|             | S4   | 1024  | 64    | 1868.7   | 3864.9    | 3.774  | 522.58  |
|             | S5   | 4096  | 64    | 5674.8   | 12411.6   | 3.030  | 688.35  |
|             | S6   | 8192  | 64    | 10775.4  | 23870.6   | 2.914  | 725.03  |
|             | S7   | 16384 | 64    | 20965.4  | 46787.7   | 2.856  | 745.28  |
|             | S8   | 65536 | 64    | 82369.2  | 184891.4  | 2.821  | 758.78  |
| BLAKE3-256  | S1   | 32    | 32    | 269.2    | 247.2     | 7.726  | 113.35  |
|             | S2   | 128   | 32    | 346.1    | 438.3     | 3.424  | 352.67  |
|             | S3   | 512   | 32    | 854.5    | 1566.9    | 3.060  | 571.44  |
|             | S4   | 1024  | 32    | 1528.5   | 3096.2    | 3.024  | 638.92  |
|             | S5   | 4096  | 32    | 3468.0   | 7443.0    | 1.817  | 1126.38 |
|             | S6   | 8192  | 32    | 3989.6   | 8615.4    | 1.052  | 1958.21 |
|             | S7   | 16384 | 32    | 8467.9   | 18684.6   | 1.140  | 1845.20 |
|             | S8   | 65536 | 32    | 35919.6  | 80446.6   | 1.228  | 1740.00 |
| Areion-MD   | S1   | 32    | 32    | 218.1    | 71.4      | 2.231  | 139.91  |
|             | S2   | 128   | 32    | 317.1    | 316.6     | 2.473  | 385.00  |
|             | S3   | 512   | 32    | 719.5    | 1207.1    | 2.358  | 678.61  |
|             | S4   | 1024  | 32    | 1254.8   | 2423.0    | 2.366  | 778.25  |

| S1–S8 test results of comparison hash algorithms on the x86 platform (continued) |      |       |       |         |           |       |        |
|----------------------------------------------------------------------------------|------|-------|-------|---------|-----------|-------|--------|
| Alg.                                                                             | Lvl. | In/B  | Out/B | Avg. ns | Avg. cyc. | cpb   | MiB/s  |
|                                                                                  | S5   | 4096  | 32    | 4544.4  | 9760.0    | 2.383 | 859.58 |
|                                                                                  | S6   | 8192  | 32    | 8830.8  | 19438.5   | 2.373 | 884.69 |
|                                                                                  | S7   | 16384 | 32    | 17484.9 | 38871.9   | 2.373 | 893.63 |
|                                                                                  | S8   | 65536 | 32    | 69235.4 | 155216.1  | 2.368 | 902.72 |

## 4.2 Hygon Platform Performance Test Results

To evaluate the implementation behavior of PAVELOR on a domestic x86-compatible processor, this subsection reports S1–S8 results on the Hygon C86-4G platform. The platform uses a Hygon C86-4G processor supporting AES and AVX2 instructions. The operating system is Linux, and the compiler is GCC/G++ 8.5.0. The optimized implementation is compiled as C99, and the benchmark program is compiled as C++11. The main optimization options are the same as those used for the x86 platform above, enabling `-O3`, AVX2, AES instructions, LTO, and frame-pointer omission.

The tests are again single-threaded and pinned to one logical core. The input lengths follow the S1–S8 classes specified by the self-evaluation guidelines:

| Level    | S1 | S2  | S3  | S4   | S5   | S6   | S7    | S8    |
|----------|----|-----|-----|------|------|------|-------|-------|
| Length/B | 32 | 128 | 512 | 1024 | 4096 | 8192 | 16384 | 65536 |

Each input length is measured 1000 times. The performance metrics are average latency, average cycle count, cycles per byte, and throughput.

Table 11 gives the results of the optimized implementation on this platform. The three output-length instances all enter relatively stable throughput regimes for medium and long messages: under the S8 input, PAVELOR-512 reaches 719.52 MB/s, PAVELOR-768 reaches 531.07 MB/s, and PAVELOR-1024 reaches 279.80 MB/s. This trend is consistent with the sponge-rate configuration of the three instances: as the output length and capacity increase, fewer message bytes are absorbed per permutation call, and the steady-state throughput decreases accordingly.

Table 11: S1–S8 test results of the optimized PAVELOR implementation on the Hygon C86-4G platform

| Alg.        | Lvl. | In/B  | Out/B | Avg. ns | Avg. cyc. | cpb    | MB/s   |
|-------------|------|-------|-------|---------|-----------|--------|--------|
| PAVELOR-512 | S1   | 32    | 64    | 401.1   | 555.4     | 17.356 | 76.08  |
|             | S2   | 128   | 64    | 396.2   | 543.4     | 4.245  | 308.08 |
|             | S3   | 512   | 64    | 944.8   | 1698.6    | 3.318  | 516.82 |
|             | S4   | 1024  | 64    | 1672.7  | 3334.5    | 3.256  | 583.82 |
|             | S5   | 4096  | 64    | 5739.5  | 12096.7   | 2.953  | 680.59 |
|             | S6   | 8192  | 64    | 11060.5 | 23541.4   | 2.874  | 706.34 |
|             | S7   | 16384 | 64    | 21950.0 | 47017.0   | 2.870  | 711.85 |
|             | S8   | 65536 | 64    | 86863.5 | 186704.5  | 2.849  | 719.52 |
| PAVELOR-768 | S1   | 32    | 96    | 416.3   | 559.2     | 17.475 | 73.31  |
|             | S2   | 128   | 96    | 610.6   | 1079.8    | 8.436  | 199.92 |
|             | S3   | 512   | 96    | 1312.7  | 2652.1    | 5.180  | 371.96 |
|             | S4   | 1024  | 96    | 2213.9  | 4766.9    | 4.655  | 441.10 |
|             | S5   | 4096  | 96    | 7814.1  | 17659.1   | 4.311  | 499.90 |
|             | S6   | 8192  | 96    | 15061.3 | 34277.2   | 4.184  | 518.71 |
|             | S7   | 16384 | 96    | 29698.1 | 67915.4   | 4.145  | 526.13 |

S1–S8 test results of the optimized PAVELOR implementation on the Hygon C86-4G platform (continued)

| Alg.         | Lvl. | In/B  | Out/B | Avg. ns  | Avg. cyc. | cpb    | MB/s   |
|--------------|------|-------|-------|----------|-----------|--------|--------|
|              | S8   | 65536 | 96    | 117687.7 | 269907.7  | 4.118  | 531.07 |
| PAVELOR-1024 | S1   | 32    | 128   | 654.5    | 1200.0    | 37.499 | 46.62  |
|              | S2   | 128   | 128   | 1045.5   | 2239.0    | 17.492 | 116.76 |
|              | S3   | 512   | 128   | 2374.9   | 5499.7    | 10.742 | 205.60 |
|              | S4   | 1024  | 128   | 4100.9   | 9820.3    | 9.590  | 238.13 |
|              | S5   | 4096  | 128   | 14564.2  | 35842.8   | 8.751  | 268.21 |
|              | S6   | 8192  | 128   | 28465.2  | 70426.5   | 8.597  | 274.46 |
|              | S7   | 16384 | 128   | 56293.5  | 139697.2  | 8.526  | 277.56 |
|              | S8   | 65536 | 128   | 223375.9 | 555212.6  | 8.472  | 279.80 |

## 5 Features of PAVELOR

- AES-instruction-friendly large-state permutation structure:** The permutation PAFF addresses the difficulty of balancing diffusion efficiency and implementation cost in a large-state setting by adopting a large-state structure well suited to AES instructions. The AES round function provides intra-subblock nonlinear transformation and local diffusion, while lightweight state-level coupling propagates information among subblocks. Compared with traditional large-state SPN structures, this design avoids explicitly introducing a heavy inter-subblock diffusion layer covering the full state. Instead, it exploits the XOR path already present during AES-instruction execution to carry part of the inter-subblock coupling, thereby achieving cross-subblock diffusion without additional heavy cost. Compared with traditional Feistel structures, which mainly rely on gradual exchange-based propagation, PAFF introduces denser cross-subblock coupling in a large state and can therefore reach full diffusion in fewer rounds. Overall, the structure preserves regularity while combining strong diffusion capability with low implementation burden, making it suitable for very large-state permutations.
- High-performance AES-instruction software:** PAVELOR is designed from the structural level around a 128-bit processing granularity, so its underlying operations map naturally to AES instructions and SIMD platforms. On general-purpose processors supporting AES instruction sets, the algorithm can fully exploit register resources and instruction-level parallelism. For the S8 long-message input and the same digest length, the optimized implementation of PAVELOR-512 is about 7.21 times faster than SHA3-512, demonstrating strong software performance.
- Scalable design for high-security applications:** PAVELOR supports three digest lengths, 512, 768, and 1024 bits, covering application requirements such as high-security authentication, long-term integrity protection, and post-quantum security reserves. It uses a unified sponge framework and derives different instances through rate and capacity parameters, so that different security levels and output specifications can be realized within the same design framework. This provides structural consistency, parameter scalability, and engineering convenience. The design targets high-security settings from the outset and is particularly suitable for applications requiring strong preimage security and long-term security margins.

## 6 Statistical Tests of PAVELOR

The statistical tests are performed on low-Hamming-weight test messages. The significance level is set to 0.010, the number of message blocks is 10000, and each test is repeated 100 times. The resulting pass rates for PAVELOR-512, PAVELOR-768, and PAVELOR-1024 are reported below. The detailed results are given in Tables 12–14.

### 6.1 Statistical Test Results for PAVELOR-512

The detailed results for PAVELOR-512 are shown in Table 12.

Table 12: Statistical test results of PAVELOR-512

| Test item                              | Pass rate (%) |
|----------------------------------------|---------------|
| Overlapping Sub-sequence (2)-1         | 98.00         |
| Overlapping Sub-sequence (2)-2         | 100.00        |
| Overlapping Sub-sequence (5)-1         | 98.00         |
| Overlapping Sub-sequence (5)-2         | 99.00         |
| Single-bit Frequency                   | 98.00         |
| Binary Derivation (3)                  | 99.00         |
| Binary Derivation (7)                  | 96.00         |
| Approximate Entropy (2)                | 99.00         |
| Approximate Entropy (5)                | 100.00        |
| Matrix Rank                            | 99.00         |
| Block Frequency (100)                  | 99.00         |
| Longest Run of Ones in a Block (10000) | 99.00         |
| Cumulative Sums                        | 98.00         |
| Discrete Fourier Transform             | 100.00        |
| Poker (4)                              | 98.00         |
| Poker (8)                              | 97.00         |
| Universal Statistical (9) (5120)       | 99.00         |
| Linear Complexity (500)                | 96.00         |
| Total Runs                             | 98.00         |
| Run Distribution                       | 97.00         |
| Autocorrelation (1)                    | 98.00         |
| Autocorrelation (2)                    | 98.00         |
| Autocorrelation (8)                    | 98.00         |
| Autocorrelation (16)                   | 97.00         |
| Message Avalanche Effect               | 99.22         |

### 6.2 Statistical Test Results for PAVELOR-768

The detailed results for PAVELOR-768 are shown in Table 13.

Table 13: Statistical test results of PAVELOR-768

| Test item                              | Pass rate (%) |
|----------------------------------------|---------------|
| Overlapping Sub-sequence (2)-1         | 99.00         |
| Overlapping Sub-sequence (2)-2         | 100.00        |
| Overlapping Sub-sequence (5)-1         | 99.00         |
| Overlapping Sub-sequence (5)-2         | 99.00         |
| Single-bit Frequency                   | 99.00         |
| Binary Derivation (3)                  | 100.00        |
| Binary Derivation (7)                  | 100.00        |
| Approximate Entropy (2)                | 100.00        |
| Approximate Entropy (5)                | 99.00         |
| Matrix Rank                            | 99.00         |
| Block Frequency (100)                  | 98.00         |
| Longest Run of Ones in a Block (10000) | 99.00         |
| Cumulative Sums                        | 100.00        |
| Discrete Fourier Transform             | 99.00         |
| Poker (4)                              | 99.00         |
| Poker (8)                              | 98.00         |
| Universal Statistical (9) (5120)       | 99.00         |
| Linear Complexity (500)                | 100.00        |
| Total Runs                             | 98.00         |
| Run Distribution                       | 99.00         |
| Autocorrelation (1)                    | 98.00         |
| Autocorrelation (2)                    | 100.00        |
| Autocorrelation (8)                    | 99.00         |
| Autocorrelation (16)                   | 99.00         |
| Message Avalanche Effect               | 99.28         |

### 6.3 Statistical Test Results for PAVELOR-1024

The detailed results for PAVELOR-1024 are shown in Table 14.

Table 14: Statistical test results of PAVELOR-1024

| Test item                      | Pass rate (%) |
|--------------------------------|---------------|
| Overlapping Sub-sequence (2)-1 | 99.00         |
| Overlapping Sub-sequence (2)-2 | 100.00        |
| Overlapping Sub-sequence (5)-1 | 97.00         |
| Overlapping Sub-sequence (5)-2 | 100.00        |
| Single-bit Frequency           | 99.00         |
| Binary Derivation (3)          | 99.00         |
| Binary Derivation (7)          | 99.00         |

Statistical test results of PAVELOR-1024 (continued)

| Test item                              | Pass rate (%) |
|----------------------------------------|---------------|
| Approximate Entropy (2)                | 99.00         |
| Approximate Entropy (5)                | 98.00         |
| Matrix Rank                            | 97.00         |
| Block Frequency (100)                  | 97.00         |
| Longest Run of Ones in a Block (10000) | 100.00        |
| Cumulative Sums                        | 100.00        |
| Discrete Fourier Transform             | 100.00        |
| Poker (4)                              | 97.00         |
| Poker (8)                              | 97.00         |
| Universal Statistical (9) (5120)       | 98.00         |
| Linear Complexity (500)                | 99.00         |
| Total Runs                             | 98.00         |
| Run Distribution                       | 99.00         |
| Autocorrelation (1)                    | 98.00         |
| Autocorrelation (2)                    | 99.00         |
| Autocorrelation (8)                    | 97.00         |
| Autocorrelation (16)                   | 99.00         |
| Message Avalanche Effect               | 99.41         |

## 7 Test Vectors of PAVELOR

This section provides representative test vectors for PAVELOR to support conformance and implementation verification. The test vectors were generated by the reference implementation; both messages and hash values are given in hexadecimal notation. The input message length of KAT\_Loop is 8192 bits. In the main text, this message is shown in abbreviated form together with the corresponding digest; the full input is available in the corresponding Test\_Vector folder.

### 7.1 PAVELOR-512

#### Example 1: KAT\_2\_12 (empty message)

```

Msg_Len = 0
Msg =
Dst_Len = 512
Dst =
C5 35 78 FA 79 56 AA D6 0E EC A5 3D 46 A2 BD 17
19 D8 05 9A D1 72 5F 2A A9 43 7B 1F 40 B5 19 F9
20 34 37 CE C8 DD A1 46 F2 90 EE 27 D5 8C 6F F2
14 CB 7B E6 AA 1D 85 A9 4F 75 0F 23 58 83 FC EA

```

**Example 2: KAT\_2\_23 (nonzero message of length  $2^{23}$  bits)**

```

Msg_Len = 8388608
Msg_Seed =
4B 41 54 5F 32 5F 32 33 4B 41 54 5F 32 5F 32 33
4B 41 54 5F 32 5F 32 33 4B 41 54 5F 32 5F 32 33
4B 41 54 5F 32 5F 32 33 4B 41 54 5F 32 5F 32 33
4B 41 54 5F 32 5F 32 33 4B 41 54 5F 32 5F 32 33
Msg_Exp = B6 E2 C8 25 BE 0E 7D 79 ...70 B7 D1 48 BC 6F CC EC
Dst_Len = 512
Dst =
B9 5B 7F 67 10 8D 2F 2F E3 2B 14 C7 9F 10 8A 7B
83 47 16 53 95 69 9D 6F 40 B7 00 8F A6 39 67 5C
8B C8 2E FD B2 D4 41 0C ED D8 EB 57 6C 19 E9 C2
E1 E8 A9 CF 32 1E 1F A4 47 20 85 09 3A 07 C9 A8

```

**Example 3: KAT\_2\_33 (nonzero message of length  $2^{33}$  bits)**

```

Msg_Len = 8589934592
Msg_Seed =
4B 41 54 5F 32 5F 33 33 4B 41 54 5F 32 5F 33 33
4B 41 54 5F 32 5F 33 33 4B 41 54 5F 32 5F 33 33
4B 41 54 5F 32 5F 33 33 4B 41 54 5F 32 5F 33 33
4B 41 54 5F 32 5F 33 33 4B 41 54 5F 32 5F 33 33
Msg_Exp = AE 64 16 F9 B6 D4 C9 7A ...31 B8 0B C6 E7 37 A9 14
Dst_Len = 512
Dst =
20 A0 6C 66 E0 D3 1C CC C4 38 BF BE 21 EF 33 43
D0 5A A8 1E BD 8B 4D 2E 5D C9 F1 01 CB 93 8A C4
F0 6A AD 19 58 52 44 C2 EF 89 E1 7E 9C 44 F2 58
D5 B9 91 46 E7 E3 9A 07 68 82 81 21 DB FE A1 6D

```

**Example 4: KAT\_Loop (fixed 8192-bit message)**

```

Msg_Len = 8192
Msg = 2D 91 18 20 A8 ED 08 79 ...E7 A8 07 DB C4 61 37 7D
Dst_Len = 512
Dst =
FA E0 27 E8 16 2E C7 21 0B 6C AC 2D D9 70 8D 4E
A3 91 67 D3 B1 4C 04 C2 AB 26 F9 96 22 96 A5 8E
EF 40 28 01 9A DC 02 4D 2D 62 35 92 52 C3 A7 99
D5 E1 10 AC 20 37 37 9B A7 29 DF 3A DA CA 5B 26

```

## 7.2 PAVELOR-768

### Example 1: KAT\_2\_12 (empty message)

```

Msg_Len = 0
Msg =
Dst_Len = 768
Dst =
65 BA F7 FF C9 D3 17 66 C7 2C 90 A5 F8 D4 2E AA
7F 36 10 60 3E B9 50 27 5F 1D A6 2B 16 78 C3 AC
B6 4F FE BF 43 31 02 92 2D 26 ED 79 E1 63 98 E8
BA 4E 46 2A ED 90 CE 33 94 AC 59 5E B3 8F BE 0B
7B A9 70 D9 8B F5 D8 7A 5C B1 4B A9 29 16 32 06
EE D7 39 08 22 0F 1F F4 4F 34 95 2E 8B E7 C4 4A

```

### Example 2: KAT\_2\_23 (nonzero message of length $2^{23}$ bits)

```

Msg_Len = 8388608
Msg_Seed =
4B 41 54 5F 32 5F 32 33 4B 41 54 5F 32 5F 32 33
4B 41 54 5F 32 5F 32 33 4B 41 54 5F 32 5F 32 33
4B 41 54 5F 32 5F 32 33 4B 41 54 5F 32 5F 32 33
4B 41 54 5F 32 5F 32 33 4B 41 54 5F 32 5F 32 33
Msg_Exp = B6 E2 C8 25 BE 0E 7D 79 ...70 B7 D1 48 BC 6F CC EC
Dst_Len = 768
Dst =
8D 38 62 C2 0E 78 DD E1 D6 D5 87 28 5E 8B C4 1F
08 79 E7 36 B4 6C FA 11 18 0A A7 C6 BC FA F7 7F
58 63 9C F6 CC AE 51 98 98 A0 82 C5 3E 53 98 61
D4 9C 57 EB 4E 03 86 F5 2B 15 16 E4 19 B3 23 13
4B 3E A0 BA A4 28 E8 61 45 EC 31 1F 53 DC DC 9C
00 D7 A2 63 70 7C 46 29 57 C0 27 02 93 BC 15 CD

```

### Example 3: KAT\_2\_33 (nonzero message of length $2^{33}$ bits)

```

Msg_Len = 8589934592
Msg_Seed =
4B 41 54 5F 32 5F 33 33 4B 41 54 5F 32 5F 33 33
4B 41 54 5F 32 5F 33 33 4B 41 54 5F 32 5F 33 33
4B 41 54 5F 32 5F 33 33 4B 41 54 5F 32 5F 33 33
4B 41 54 5F 32 5F 33 33 4B 41 54 5F 32 5F 33 33
Msg_Exp = AE 64 16 F9 B6 D4 C9 7A ...31 B8 0B C6 E7 37 A9 14
Dst_Len = 768
Dst =

```

```

1B 12 64 8E 47 9D 07 61 42 1A F5 4D E6 7C 4A 2B
BA 1D 75 5C 34 24 E1 5C 5C 81 EE 2D E7 DE 82 45
96 79 32 98 5A 23 1E 84 75 18 85 D3 7A AB 5C 0A
C0 D9 9A 23 03 63 A3 54 A6 5C 8C 79 AC 68 D9 44
1D 90 B3 87 DA A1 C3 40 78 03 B7 D2 0F 8D A1 2C
F3 39 29 B9 4B 86 BB 6F 54 59 02 6A 4B 8B 8A 7E

```

#### Example 4: KAT\_Loop (fixed 8192-bit message)

```

Msg_Len = 8192
Msg = 2D 91 18 20 A8 ED 08 79 ...E7 A8 07 DB C4 61 37 7D
Dst_Len = 768
Dst =
E8 63 0B D8 B4 FB 00 BF 64 D3 A2 11 DE A2 00 08
2E EE 80 FD DC EE 70 E5 F3 5B 1F B6 2D 79 1E AB
3A A3 05 98 46 FF 41 3B 61 6C 45 24 EF 32 F6 C9
C2 4F 56 7D 83 D5 34 A8 B1 17 59 C6 CF 60 CC 23
69 AF 4A 0C E9 7C FD A2 CF 53 B3 84 CC 75 26 C1
17 DB AE 36 E6 D8 57 7A F7 35 FC AE 06 3B 20 CF

```

### 7.3 PAVELOR-1024

#### Example 1: KAT\_2\_12 (empty message)

```

Msg_Len = 0
Msg =
Dst_Len = 1024
Dst =
70 11 CC 8C 8B D1 1A C9 71 BB 39 4C BD B5 26 93
62 FA CF 05 B7 40 D0 A4 CB 1F 14 C0 A9 1A E7 F0
0E B7 A4 7D BD 96 EB BC 1B 07 35 F1 C5 A3 7E 74
5C 30 53 68 2B 5D 73 3F F1 04 5D 90 A7 26 9D 17
6B 44 8D 00 58 45 5B 75 0A B2 EF 38 AE 7C F8 9F
35 EB 4D 0E 98 D1 C8 64 E5 1F DA 93 D8 2A D8 04
A8 EA 8C DE F7 6F CB 29 AF F2 98 8F 0C 3B 3F 2C
79 19 93 94 93 E5 71 4F 47 0A BB 02 C3 75 13 63

```

#### Example 2: KAT\_2\_23 (nonzero message of length $2^{23}$ bits)

```

Msg_Len = 8388608
Msg_Seed =
4B 41 54 5F 32 5F 32 33 4B 41 54 5F 32 5F 32 33
4B 41 54 5F 32 5F 32 33 4B 41 54 5F 32 5F 32 33
4B 41 54 5F 32 5F 32 33 4B 41 54 5F 32 5F 32 33

```

```

4B 41 54 5F 32 5F 32 33 4B 41 54 5F 32 5F 32 33
Msg_Exp = B6 E2 C8 25 BE 0E 7D 79 ...70 B7 D1 48 BC 6F CC EC
Dst_Len = 1024
Dst =
18 93 09 6F F9 15 09 DF FE F0 80 87 18 E7 18 10
79 5B 66 2C 11 C6 4D 68 73 FA 0D 34 38 BD C1 85
1D 28 A1 40 2B FE 31 21 98 F6 C6 6A F9 1F 75 5D
A5 B7 A1 6F 36 98 77 8E 15 A2 90 5F AE DA 3A F0
9F 6F 44 93 EE 49 6E EF 91 8E 81 0A F8 78 FE 30
1E FD B8 AB D5 BD 68 33 95 67 1F B6 F9 29 32 61
E5 21 56 68 2D 31 E3 DE 62 C8 93 27 5D BD 54 2A
E3 3B 0F CA EE 1E BC 45 3D 10 2C 31 A5 B1 90 D8

```

### Example 3: KAT\_2\_33 (nonzero message of length $2^{33}$ bits)

```

Msg_Len = 8589934592
Msg_Seed =
4B 41 54 5F 32 5F 33 33 4B 41 54 5F 32 5F 33 33
4B 41 54 5F 32 5F 33 33 4B 41 54 5F 32 5F 33 33
4B 41 54 5F 32 5F 33 33 4B 41 54 5F 32 5F 33 33
4B 41 54 5F 32 5F 33 33 4B 41 54 5F 32 5F 33 33
Msg_Exp = AE 64 16 F9 B6 D4 C9 7A ...31 B8 0B C6 E7 37 A9 14
Dst_Len = 1024
Dst =
BD 48 A8 82 BC EA FD 3B C5 65 F7 BD 55 8E A6 97
7A 8F CE 93 CC 9B 5B 36 A3 2B A2 2D B9 20 8C A7
9B 22 07 C7 69 40 0F CC 1F F9 CD B1 79 63 04 72
CD 47 5B B5 BB BB DB 90 F0 79 A9 94 82 F1 FE 29
95 4E 9C 6F 58 90 71 58 34 D0 F0 4E B9 77 DF 35
2F 74 34 DC 92 AF 46 20 2E BA 12 C4 90 DC 9C E9
48 9D 0E 40 70 B0 AE B5 37 A9 9B FC 69 E5 31 19
33 EF 88 00 1C 71 AB D5 2D F5 FB 5D 8E E2 A8 BA

```

### Example 4: KAT\_Loop (fixed 8192-bit message)

```

Msg_Len = 8192
Msg = 2D 91 18 20 A8 ED 08 79 ...E7 A8 07 DB C4 61 37 7D
Dst_Len = 1024
Dst =
C0 69 3F 86 D1 41 15 D0 13 7D 8F A9 44 B2 D3 7C
06 65 4F 58 A5 03 5C 4B 09 65 98 47 08 72 E6 29
BA 31 30 5B DA 64 22 F5 36 0F 8E 51 DC 16 E4 05

```

8C 4C 43 63 FA AA 85 CE 48 0B BC 0F 6E 6E 86 87  
46 C1 F6 7B BB A6 78 66 C2 AC C5 BA 9C D3 67 6D  
69 73 BF 4A F8 3F 69 B5 7E E5 64 4D 74 AD 0A E4  
8F 82 63 51 89 4A DF 09 DB FD AA 63 E8 3D 66 08  
13 74 7F 00 AD B1 C3 25 0A F4 0E C3 D5 04 A8 D0

## References

- [1] Advanced Micro Devices. *AMD64 Architecture Programmer's Manual, Volume 4: 128-Bit, 256-Bit, and 512-Bit Media Instructions*, 2026.
- [2] Gorjan Alagic, Joseph Carolan, Christian Majenz, and Saliha Tokat. The sponge is quantum indifferentiable. arXiv:2504.16887, 2025.
- [3] Kazumaro Aoki and Yu Sasaki. Meet-in-the-middle preimage attacks against reduced SHA-0 and SHA-1. In Shai Halevi, editor, *Advances in Cryptology – CRYPTO 2009*, volume 5677 of *Lecture Notes in Computer Science*, pages 70–89. Springer, 2009.
- [4] Arm Ltd. The cryptographic extension in aarch64 state, 2026. Arm Developer documentation.
- [5] Zhenzhen Bao, Xiaoyang Dong, Jian Guo, Zheng Li, Danping Shi, Siwei Sun, and Xiaoyun Wang. Automatic search of meet-in-the-middle preimage attacks on aes-like hashing. In *Advances in Cryptology – EUROCRYPT 2021*, volume 12696 of *Lecture Notes in Computer Science*, pages 771–804. Springer, 2021.
- [6] Navid Ghaedi Bardeh and Sondre Rønjom. The exchange attack: How to distinguish six rounds of aes with  $2^{88.2}$  chosen plaintexts. In *Advances in Cryptology – ASIACRYPT 2019, Part III*, volume 11923 of *Lecture Notes in Computer Science*, pages 347–370. Springer, 2019.
- [7] Ryad Benadjila, Olivier Billet, Henri Gilbert, Gilles Macario-Rat, Thomas Peyrin, Matt Robshaw, and Yannick Seurin. SHA-3 proposal: ECHO, 2008. Submission to the NIST SHA-3 Competition.
- [8] Guido Bertoni, Joan Daemen, Micha"el Peeters, and Gilles Van Assche. On the indistinguishability of the sponge construction. In *Advances in Cryptology – EUROCRYPT 2008*, volume 4965 of *Lecture Notes in Computer Science*, pages 181–197. Springer, 2008.
- [9] Guido Bertoni, Joan Daemen, Micha"el Peeters, and Gilles Van Assche. Cryptographic sponge functions. Keccak Team report, 2011.
- [10] Eli Biham, Alex Biryukov, and Adi Shamir. Miss in the middle attacks on idea and khufu. In *Fast Software Encryption – FSE 1999*, Lecture Notes in Computer Science. Springer, 1999.
- [11] Victor Cauchois, Clément Gomez, and Gaël Thomas. General diffusion analysis: How to find optimal permutations for generalized type-II feistel schemes. *IACR Transactions on Symmetric Cryptology*, 2019(1):264–301, 2019.

- [12] Jiali Choy, Huihui Yap, Khoongming Khoo, Jian Guo, Thomas Peyrin, Axel Poschmann, and Chik How Tan. SPN-hash: Improving the provable resistance against differential collision attacks. In Aikaterini Mitrokotsa and Serge Vaudenay, editors, *Progress in Cryptology – AFRICACRYPT 2012*, volume 7374 of *Lecture Notes in Computer Science*, pages 270–286, Berlin, Heidelberg, 2012. Springer.
- [13] Jan Czajkowski, Leon Groot Bruinderink, Andreas Hülsing, Christian Schaffner, and Dominique Unruh. Post-quantum security of the sponge construction. *Cryptology ePrint Archive*, Paper 2017/771, 2017.
- [14] Joan Daemen and Vincent Rijmen. The wide trail design strategy. In Bahram Honary, editor, *Cryptography and Coding*, volume 2260 of *Lecture Notes in Computer Science*, pages 222–238, Berlin, Heidelberg, 2001. Springer.
- [15] Joan Daemen and Vincent Rijmen. *The Design of Rijndael: AES – The Advanced Encryption Standard*. Springer, 2002.
- [16] Patrick Derbez, Pierre-Alain Fouque, Baptiste Lambin, and Victor Mollimard. Efficient search for optimal diffusion layers of generalized feistel networks. *IACR Transactions on Symmetric Cryptology*, 2019(2):218–240, 2019.
- [17] Xiaoyang Dong, Jialiang Hua, Siwei Sun, Zheng Li, Xiaoyun Wang, and Lei Hu. Meet-in-the-middle attacks revisited: Key-recovery, collision, and preimage attacks. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021*, volume 12827 of *Lecture Notes in Computer Science*, pages 278–308. Springer, 2021.
- [18] Xiaoyang Dong, Boxin Zhao, Lingyue Qin, Qingliang Hou, Shun Zhang, and Xiaoyun Wang. Generic mitm attack frameworks on sponge constructions. In *Advances in Cryptology – CRYPTO 2024, Part IV*, volume 14923 of *Lecture Notes in Computer Science*, pages 3–37. Springer, 2024.
- [19] Praveen Gauravaram, Lars R. Knudsen, Krystian Matusiewicz, Florian Mendel, Christian Rechberger, Martin Schläffer, and Søren S. Thomsen. Grøstl – a SHA-3 candidate. In Helena Handschuh, Stefan Lucks, Bart Preneel, and Phillip Rogaway, editors, *Symmetric Cryptography*, volume 9031 of *Dagstuhl Seminar Proceedings*, pages 1–33, Dagstuhl, Germany, 2009. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [20] Shay Gueron. *Intel® Advanced Encryption Standard (AES) New Instructions Set*. Intel Corporation, 2010.
- [21] Shay Gueron and Nicky Mouha. Simpira v2: A family of efficient permutations using the aes round function. In *Advances in Cryptology – ASIACRYPT 2016, Part I*, volume 10031 of *Lecture Notes in Computer Science*, pages 95–125. Springer, 2016.
- [22] Jian Guo, San Ling, Christian Rechberger, and Huaxiong Wang. Advanced meet-in-the-middle preimage attacks: First results on full Tiger, and improved results on MD4 and SHA-2. In Masayuki Abe, editor, *Advances in Cryptology – ASIACRYPT 2010*, volume 6477 of *Lecture Notes in Computer Science*, pages 56–75. Springer, 2010.
- [23] Intel Corporation. *Intel® 64 and IA-32 Architectures Optimization Reference Manual*, 2024.

- [24] Takanori Isobe, Ryoma Ito, Fukang Liu, Kazuhiko Minematsu, Motoki Nakahashi, Kosei Sakamoto, and Rentaro Shiba. Areion: Highly-efficient permutations and its applications to hash functions for short input. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(2):115–154, 2023.
- [25] Stefan K olbl, Martin M. Lauridsen, Florian Mendel, and Christian Rechberger. Haraka v2 – efficient short-input hashing for post-quantum applications. *IACR Transactions on Symmetric Cryptology*, 2016(2):1–29, 2016.
- [26] Nicky Mouha, Qingju Wang, Dawu Gu, and Bart Preneel. Differential and linear cryptanalysis using mixed-integer linear programming. In *Information Security and Cryptology – Inscrypt 2011*, volume 7537 of *Lecture Notes in Computer Science*, pages 57–76. Springer, 2012.
- [27] National Institute of Standards and Technology. *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. NIST, 2015.
- [28] National Institute of Standards and Technology. *Advanced Encryption Standard (AES)*. NIST, 2023.
- [29] Ivica Nikoli . Tiaoxin-346, version 1.0. CAESAR first-round submission, 2014.
- [30] Jack O’Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O’Hearn. BLAKE3: One function, fast everywhere. Specification and design rationale, 2020.
- [31] Lingyue Qin, Jialiang Hua, Xiaoyang Dong, Hailun Yan, and Xiaoyun Wang. Meet-in-the-middle preimage attacks on sponge-based hashing. In *Advances in Cryptology – EUROCRYPT 2023, Part IV*, volume 14007 of *Lecture Notes in Computer Science*, pages 158–188. Springer, 2023.
- [32] Kosei Sakamoto, Fukang Liu, Yuto Nakano, Shinsaku Kiyomoto, and Takanori Isobe. Rocca: An efficient aes-based encryption scheme for beyond 5g (full version). Cryptology ePrint Archive, Paper 2022/116, 2022.
- [33] Kosei Sakamoto, Kazuhiko Minematsu, Nao Shibata, Maki Shigeri, Hiroyasu Kubo, and Takanori Isobe. Design of a linear layer for a block cipher based on type-2 generalized feistel network with 32 branches. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, E105.A(3):278–288, 2022.
- [34] Kyoji Shibutani and Tetsu Iwata. On the (im)possibility of improving the round diffusion of generalized feistel structures. *Information Processing Letters*, 174:106197, 2022.
- [35] Tomoyasu Suzaki and Kazuhiko Minematsu. Improving the generalized feistel. In *Fast Software Encryption – FSE 2010*, volume 6147 of *Lecture Notes in Computer Science*, pages 19–39. Springer, 2010.
- [36] Tomoyasu Suzaki, Kazuhiko Minematsu, Sumio Morioka, and Eita Kobayashi. TWINE: A lightweight block cipher for multiple platforms. In Lars R. Knudsen and Huapeng Wu, editors, *Selected Areas in Cryptography – SAC 2012*, volume 7707 of *Lecture Notes in Computer Science*, pages 339–354, Berlin, Heidelberg, 2013. Springer.

- [37] Yosuke Todo. Structural evaluation by generalized integral property. In *Advances in Cryptology – EUROCRYPT 2015*, volume 9056 of *Lecture Notes in Computer Science*, pages 287–314. Springer, 2015.
- [38] Yosuke Todo and Masakatu Morii. Bit-based division property and application to simon family. In *Fast Software Encryption – FSE 2016*, volume 9783 of *Lecture Notes in Computer Science*, pages 357–377. Springer, 2016.
- [39] Hongjun Wu and Bart Preneel. Aegis: A fast authenticated encryption algorithm. In *Selected Areas in Cryptography – SAC 2013*, volume 8282 of *Lecture Notes in Computer Science*, pages 185–201. Springer, 2013.
- [40] Zejun Xiang, Wentao Zhang, Zhenzhen Bao, and Dongdai Lin. Applying milp method to searching integral distinguishers based on division property for 6 lightweight block ciphers. In *Advances in Cryptology – ASIACRYPT 2016, Part I*, volume 10031 of *Lecture Notes in Computer Science*, pages 648–678. Springer, 2016.